Compiladores (CC3001) — Resumo Teórico

Gonçalo Sousa Atualizado: Semana 5 (Aulas 1–7)

Conteúdo

Aula 1 — Introdução e fases de um compilador **2**
O que é um compilador . . . 2
Fases de um compilador . . . 2
Aula 2 — Análise lexical **3**
Tokens: o que o analisador lexical produz . . . 3
Expressões regulares . . . 3
Autómatos finitos: DFA e NFA . . . 5
De expressão regular a DFA . . . 5
Aula 3 — Geradores de analisadores lexicais **9**
Implementação direta de um DFA em C . . . 9
Geradores: Alex e Flex . . . 10
Que regra ganha: *longest match* e *first match* . . . 11
Aula 4 — Análise sintática e gramáticas independentes de contexto **13**
Gramáticas independentes de contexto . . . 13
Linguagem de uma gramática e como escrever uma . . . 14
Árvores sintáticas e ambiguidade . . . 15
Eliminar ambiguidade: associatividade e precedência . . . 16
O problema do "dangling else" . . . 18
Aula 5 — Análise sintática *bottom-up* (LR) **19**
Análise *bottom-up*: shift e reduce . . . 19
Tabela LR e o algoritmo de *parsing* . . . 20
Autómato e tabela LR(0) . . . 22
Análise SLR(1) . . . 26
Conflitos e como resolvê-los . . . 29
Extras: LR(1) e LALR(1) . . . 30
Aula 6 — Geradores de analisadores sintáticos e sintaxe abstrata **31**
Geradores de analisadores LR . . . 31
Happy . . . 31
Ações semânticas . . . 32
Bison . . . 34
Conflitos, associatividade e precedência . . . 35
Sintaxe concreta e abstrata . . . 37
Representar sintaxe abstrata . . . 38
Aula 7 — Análise sintática *top-down* (LL) **44**
Descida recursiva . . . 44
Gramáticas LL(1) e *parsing* preditivo . . . 46

O sinal † marca matéria que não está nos slides mas faz falta para as práticas.

1 Aula 1
Introdução e fases de um compilador

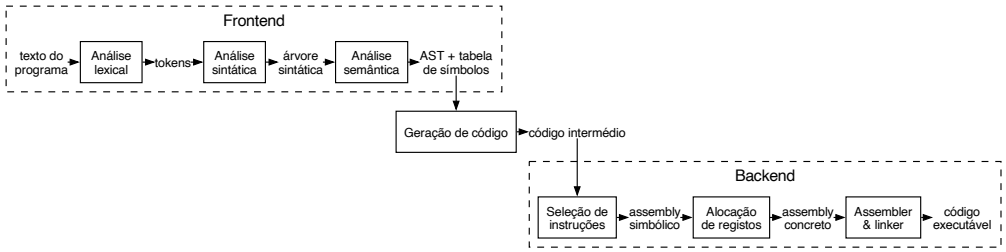
O que é um compilador

Um **compilador** traduz um programa de uma **linguagem fonte** para um programa equivalente numa **linguagem destino**, quase sempre de alto para baixo nível (C → código máquina, Java → *bytecode* da JVM). O destino também pode ser de alto nível (Elm → JavaScript). Um **interpretador** faz a mesma análise lexical e sintática, mas **executa logo** o programa (a partir da árvore ou de código intermédio) e não produz um executável.

	Vantagem
Interpretação	Mais simples de implementar; corre em várias arquiteturas com facilidade; permite um REPL (*read-eval-print-loop*).
Compilação	Código normalmente mais eficiente; o executável distribui-se sem o compilador.

Muitas implementações têm as duas coisas (o GHC tem `ghc` e `ghci`). Um compilador é **self-hosted** se está escrito na linguagem que compila (GCC em C); a primeira versão faz-se por **bootstrapping**: um compilador escrito noutra linguagem compila a versão escrita na própria linguagem.

Fases de um compilador



Definição **Frontend e backend**

- **Frontend** (depende da linguagem fonte, não da máquina): análise lexical → análise sintática → análise semântica. Produz a AST e a tabela de símbolos.
- **Backend** (depende da máquina, não da linguagem): seleção de instruções → alocação de registos → *assembler* e *linker*.
- Entre os dois, o **código intermédio**, que não depende de nenhum.

A separação permite **reutilizar** metades: o mesmo *backend* serve várias linguagens (o LLVM serve Clang, Swift e Rust). A cadeira trata do **frontend**, uma fase por aula:

Fase	Entra	Sai	Aulas
Análise lexical	carateres	*tokens*	2–3
Análise sintática	*tokens*	árvore (AST)	4–7

2 Aula 2
Análise lexical

Tokens: o que o analisador lexical produz

O **analisador lexical** (*lexer*, *scanner*) parte o texto numa sequência de ***tokens***, para que o analisador sintático trabalhe com "um identificador, um +, um número" e não com carateres soltos. Os *tokens* de uma linguagem (o seu **léxico**) são **identificadores**, **literais** (números, carateres, cadeias), **palavras reservadas** (`if`, `while`) e **operadores**.

Definição **Representação de um token**

Cada *token* tem uma **etiqueta** e, nalguns, um **valor**:

Entrada	Etiqueta	Valor
`if`	`IF`	—
`,`	`COMMA`	—
`!=`	`NOTEQ`	—
`main`	`ID`	`"main"`
`71`	`NUM`	`71`
`.5`	`REAL`	`0.5`

Espaços, mudanças de linha e comentários **não dão *tokens***: só separam *tokens*. O `EOF` marca o fim da entrada.

Exemplo resolvido

`float match0(char *s) { if (!strncmp(s, "0.0", 3)) return 0.0; }` dá:
`FLOAT, ID("match0"), LPAREN, CHAR, STAR, ID("s"), RPAREN, LBRACE, IF, LPAREN, BANG, ID("strncmp"), LPAREN, ID("s"), COMMA, STRING("0.0"), COMMA, NUM(3), RPAREN, RPAREN, RETURN, REAL(0.0), SEMI, RBRACE, EOF`

O caminho para construir um *lexer* sem o escrever à mão é sempre o mesmo, e é o resto desta aula e da seguinte:

1. descrever cada *token* com uma **expressão regular** (o quê);
2. traduzir a expressão num **NFA** (construção de Thompson);
3. converter o NFA num **DFA** (construção de subconjuntos) e minimizá-lo;
4. implementar o DFA, ou deixar um **gerador** (Alex, Flex) fazer 2–4.

Expressões regulares

Definição **Expressões regulares**

Dado um alfabeto Σ, uma **linguagem** é um conjunto $L \subseteq \Sigma^*$ de palavras. As expressões regulares são:

- a — uma letra $a \in \Sigma$ (reconhece só `"a"`);
- ε — a palavra vazia;
- $M \mid N$ — **união**: o que M ou N reconhecem;
- $M \cdot N$ — **concatenação**: uv com $u \in L(M)$ e $v \in L(N)$;
- M^* — **repetição** (fecho de Kleene): zero ou mais palavras de M.

A concatenação escreve-se sem ponto (abc). **Prioridade**: repetição > concatenação > união.

Erro típico: ler mal a prioridade. $ab|c$ é $(ab)|c$, não $a(b|c)$; e ab^* é $a(b^*)$, não $(ab)^*$. Na dúvida, põe parêntesis.

Abreviatura	Significado
`M?`	opcional: $M \mid \varepsilon$
`M+`	uma ou mais vezes: MM^*
`[x1x2...xn]`	um dos carateres: $x_1 \| x_2 \| \ldots \| x_n$
`[a-z]`	intervalo de carateres
`[^a-z]`	qualquer carácter fora do intervalo
`.`	qualquer carácter exceto mudança de linha
`"a+"`	cadeia literal (o + não é operador)

Escrever a expressão de um *token* †

Pergunta, por esta ordem:

1. **Por que carácter pode começar?** Dá a primeira classe (`[_a-zA-Z]`).
2. **O que se repete, e pode faltar?** $*$ se pode faltar, `+` se não, `?` para uma parte que aparece no máximo uma vez.
3. **Há partes opcionais que não podem faltar as duas?** Separa em alternativas. Em `[0-9]*"."[0-9]*` os dois lados são opcionais, por isso aceita `"."` sozinho; daí a forma do `REAL` abaixo.
4. **Há um terminador** ($*/$, `"`)? Proíbe-o no meio (ver a seguir).
5. **Testa** com palavras que devem passar, com as que não devem, e com a palavra vazia.

Exemplo resolvido

Os *tokens* de um pequeno analisador em C:

```
if                                  palavra reservada (IF)
[_a-zA-Z][_a-zA-Z0-9]*              identificadores (ID)
[0-9]+                              número inteiro (NUM)
([0-9]+"."[0-9]*)|([0-9]*"."[0-9]+) número fracionário (REAL)
//.*                                comentário até ao fim da linha
```

`REAL`: ou há dígitos antes do ponto (`12.`, `12.5`), ou há dígitos depois (`.5`); `"."` sozinho não passa em nenhuma das duas. A expressão dos identificadores também reconhece `if`: quem ganha decide-se na Aula 3 (*first match*).

Parar no primeiro terminador. Para um comentário `/* ... */`, a expressão `"/*" .* "*/"` **está errada**: `.*` consome o máximo possível, e em `/* a */ x /* b */` apanha tudo como **um** comentário, do primeiro `/*` ao último `*/`. A técnica é proibir o terminador no meio. Para `<!-- ... -->`: `"<!--" ([^-]|"-"[^-])* "-->"` (qualquer carácter que não seja –, ou um – que não seja seguido de outro –). Para `*/` a ideia é a mesma.

Pratica agora **Expressões regulares (Folha lab. 2, Exercício 2)**

Papel e lápis, com as perguntas acima:

- **(a)** — expressões regulares para todos os tokens do `scanner-c` (`ID`, `NUM`, `LPAREN`, `RPAREN`, `COMMA`, `IF` e os do Exercício 1: chavetas, `;`, `WHILE`, `FOR`, `INT`, `FLOAT`, `REAL`). Não te esqueças do _ nos identificadores.
- **(b)** — generaliza `REAL` para notação científica (`12.34e+12`, `1e-12`, `123.4E+9`): repara que em `1e-12` não há ponto.
- **(c)** — comentários `// ...` e `/* ... */`. Para o multi-linha usa a técnica "parar no primeiro terminador", adaptada a `*/`.

Soluções: Ex. 2 (a), Ex. 2 (b), Ex. 2 (c)

Autómatos finitos: DFA e NFA

Uma expressão regular diz **o que** se reconhece; um **autómato finito** diz **como**: lê a palavra um carácter de cada vez, muda de estado em cada um, e no fim vê se parou num estado final.

Definição **DFA (autómato finito determinístico)**

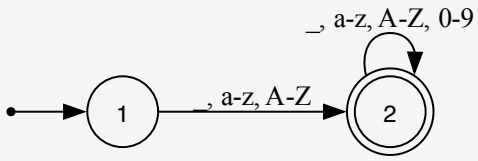
Um quíntuplo $(\Sigma, Q, q_0, F, \delta)$:

- Σ — alfabeto; Q — conjunto finito de estados;
- $q_0 \in Q$ — estado inicial; $F \subseteq Q$ — estados finais;
- $\delta \subseteq Q \times \Sigma \times Q$ — transições, com **no máximo uma** saída por cada estado q e símbolo σ (uma função parcial $q' = \delta(q, \sigma)$).

Uma palavra é **aceite** se, partindo de q_0 e seguindo as transições dos seus símbolos, se acaba num estado de F. Se faltar uma transição a meio, a palavra é **rejeitada**.

Exemplo resolvido

DFA de `[_a-zA-Z][_a-zA-Z0-9]*` (identificadores):



$Q = \{1, 2\}$, $q_0 = 1$, $F = \{2\}$; δ tem $(1, \ell, 2)$ para cada letra ou _ ℓ, e $(2, \ell, 2)$, $(2, d, 2)$ para cada letra ou _ ℓ e dígito d. Para `x1`: $1 \xrightarrow{x} 2 \xrightarrow{1} 2$, e $2 \in F$: **aceite**. Para `1x`: o estado 1 não tem transição com `1`: **rejeitada**.

Definição **NFA (autómato finito não-determinístico)**

Igual a um DFA, mas com $\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times Q$:

- pode haver **várias transições** com o mesmo símbolo a partir do mesmo estado;
- há **transições-ε**, que mudam de estado **sem consumir** nenhum símbolo.

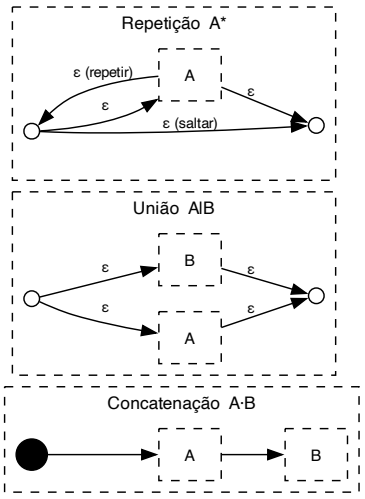
Aceita se **algum** caminho leva a um estado final.

Um NFA é fácil de construir a partir de uma expressão regular, mas não se implementa diretamente (pode estar em vários estados ao mesmo tempo); um DFA implementa-se com uma tabela. Por isso o caminho é **expressão → NFA → DFA**.

De expressão regular a DFA

Construção de Thompson: de expressão regular a NFA

Cada sub-expressão vira um **fragmento** de autómato com **uma entrada e uma saída**, e os fragmentos encaixam-se seguindo a estrutura da expressão.



Definição **Regras de Thompson**

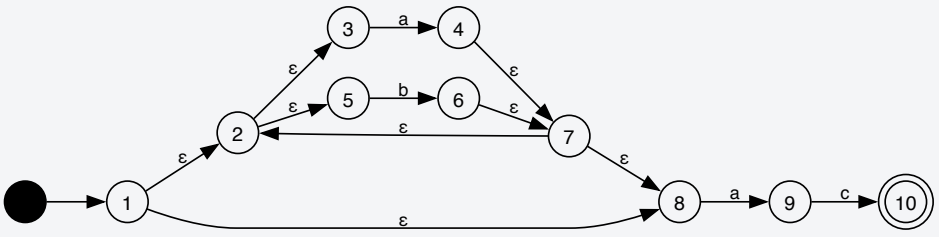
- **Símbolo** σ (ou ε): uma transição σ da entrada para a saída.
- **Concatenação** AB: a saída de A liga à entrada de B.
- **União** $A \mid B$: entrada nova com ε para as entradas de A e B; as saídas de A e B ligam por ε a uma saída nova.
- **Repetição** A^*: entrada e saída novas. A entrada liga por ε à entrada de A e **diretamente à saída** (zero repetições); a saída de A liga por ε à entrada de A (repetir) e à saída nova (terminar).

Procedimento: (1) parte a expressão pelos operadores, respeitando a prioridade (o último a aplicar é a raiz); (2) constrói os fragmentos de dentro para fora, dos símbolos para a raiz; (3) numera os estados no fim. **Erro típico:** no A^*, esquecer a ligação entrada → saída; sem ela o autómato exige pelo menos uma repetição (fica A^+).

Exemplo resolvido

NFA de $(a \mid b)^*ac$ (o exemplo da aula). A raiz é a concatenação $((a|b)^*)\,a\,c$:

1. Fragmento `a` (estados 3→4) e fragmento `b` (5→6).
2. **União** `a|b`: estado 2 (entrada, ε para 3 e 5) e estado 7 (saída, ε a partir de 4 e 6).
3. **Repetição** `(a|b)*`: par 1 (entrada) / 8 (saída); 1 → 2 (entrar), 1 → 8 (zero repetições), 7 → 2 (repetir), 7 → 8 (terminar).
4. **Concatenação** com `a`: $8 \xrightarrow{a} 9$.
5. **Concatenação** com `c`: $9 \xrightarrow{c} 10$ (final).



Construção de subconjuntos: de NFA a DFA

A ideia: o DFA segue **todos os caminhos do NFA ao mesmo tempo**. Cada estado do DFA é o **conjunto** dos estados do NFA onde se pode estar.

> **Definição** **closure e construção de subconjuntos**
>
> Para um NFA $(\Sigma, Q, q_0, F, \delta)$:
>
> $$\text{closure}(S) = \{\, q' \,:\, \text{há } q \in S \text{ e um caminho só de transições-}\varepsilon \text{ de } q \text{ a } q' \,\}$$
>
> (os estados a que se chega a partir de S **sem consumir símbolos**; inclui os de S). O DFA tem:
>
> $$S_0 = \text{closure}(\{q_0\}) \qquad \delta'(S,\sigma) = \text{closure}(\text{move}(S,\sigma)) \qquad F' = \{\, S \,:\, S \cap F \neq \varnothing \,\}$$
>
> com $\text{move}(S,\sigma) = \{\, q' \,:\, q \in S,\ (q,\sigma,q') \in \delta \,\}$.

Procedimento (para cada estado do DFA ainda não tratado, e cada símbolo σ):

1. **Que estados de S têm uma transição com σ?** Os destinos são o move.
2. **Closure do move:** junta tudo o que se alcança por ε, e repete a partir dos que entraram.
3. **O conjunto é vazio?** Não há transição. **Já existe?** Liga-o. **É novo?** É um estado novo, a tratar depois.
4. **É final?** Se contém algum estado final do NFA.

Para quando nenhum estado novo aparece.

Erros típicos: fazer o closure só um nível (de 4 → 7 também se segue 7 → 2 e 2 → 3); esquecer que o closure inclui os próprios estados do move (o 9 abaixo); fazer o move a partir de estados que não estão em S.

Exemplo resolvido

Subconjuntos para o NFA de $(a \mid b)^*ac$. Transições-ε: 1→2, 1→8, 2→3, 2→5, 4→7, 6→7, 7→2, 7→8. Com símbolo: $3 \xrightarrow{a} 4$, $5 \xrightarrow{b} 6$, $8 \xrightarrow{a} 9$, $9 \xrightarrow{c} 10$.

Estado inicial: $S_0 = \text{closure}(\{1\}) = \{1,2,3,5,8\}$ (de 1: 1 → 2, 2 → 3, 2 → 5, 1 → 8; nenhum destes tem mais ε).

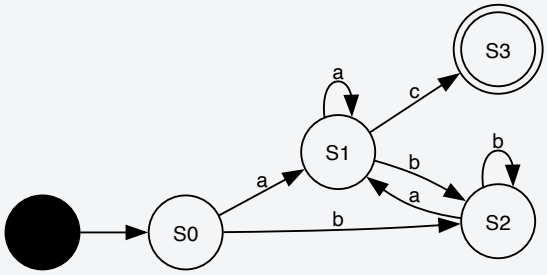
- S_0 **com a**: em S_0 só 3 e 8 têm a, move = $\{4,9\}$. closure($\{4,9\}$): 4 → 7, 7 → 2, 7 → 8, 2 → 3, 2 → 5; o 9 não tem ε. Dá $\{2,3,4,5,7,8,9\}$, novo: S_1.
- S_0 **com b**: só 5 tem b, move = $\{6\}$. closure($\{6\}$): 6 → 7, 7 → 2, 7 → 8, 2 → 3, 2 → 5. Dá $\{2,3,5,6,7,8\}$, novo: S_2.
- S_0 **com c**: só o 9 tem c, e $9 \notin S_0$: sem transição.

Exemplo resolvido

Continuação, com $S_1 = \{2, 3, 4, 5, 7, 8, 9\}$ e $S_2 = \{2, 3, 5, 6, 7, 8\}$:

- S_1 **com a**: 3 e 8 $\in S_1$, move = $\{4, 9\}$, closure = S_1: ciclo em a.
- S_1 **com b**: 5 $\in S_1$, move = $\{6\}$, closure = S_2.
- S_1 **com c**: 9 $\in S_1$, move = $\{10\}$, closure($\{10\}$) = $\{10\}$, novo: S_3. Contém o final 10, logo $S_3 \in F'$.
- S_2 **com a**: 3 e 8 $\in S_2$ (o 8 entrou pelo closure, $7 \to 8$), move = $\{4, 9\}$, closure = S_1.
- S_2 **com b**: 5 $\in S_2$, move = $\{6\}$, closure = S_2: ciclo em b.
- S_2 **com c**: o 9 não está em S_2: sem transição.
- S_3: o 10 não tem transições: nenhuma.

Não aparecem estados novos: o DFA tem S_0 a S_3, com $F' = \{S_3\}$.



Minimização

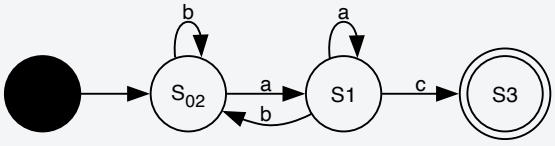
O DFA dos subconjuntos pode ter **estados equivalentes**, que se comportam da mesma maneira para qualquer resto da entrada. Fundindo-os obtém-se o **autómato mínimo**; os geradores (Alex, Flex) fazem isto sozinhos. Para verificar à mão que dois estados p e q são equivalentes †:

1. São **os dois finais**, ou **os dois não-finais**?
2. Para **cada** símbolo, vão os dois para o mesmo estado (ou para estados equivalentes), ou nenhum tem transição?

Se sim às duas, fundem-se. **Erro típico:** comparar os conjuntos de estados do NFA (S_0 e S_2 abaixo são conjuntos diferentes); o que conta é o **comportamento** no DFA.

Exemplo resolvido

No DFA acima, S_0 e S_2: (1) nenhum é final; (2) com a vão os dois para S_1, com b os dois para S_2, e com c nenhum tem transição. Logo $S_0 \equiv S_2$. Os outros pares não se fundem: S_3 é o único final, e S_1 é o único com transição em c. O autómato mínimo tem 3 estados:



Lê-se: "S_{02} = o último símbolo não foi um a; S_1 = o último foi um a; S_3 = acabou em ac".

3 Aula 3
Geradores de analisadores lexicais

O último passo do caminho expressão → NFA → DFA: **implementar** o DFA, à mão ou com um **gerador** que faz tudo a partir das expressões regulares.

Implementação direta de um DFA em C

Um DFA implementa-se com uma **tabela de transições** `delta[estado][símbolo]` e um ciclo que lê um carácter de cada vez e consulta a tabela.

Exemplo resolvido

DFA dos identificadores, com o estado `0` a representar "erro" e o estado final `2`:

```c
#define STATES  3    // estados 0 (erro), 1, 2
#define SYMBOLS 128  // códigos ASCII
const int delta[STATES][SYMBOLS] = {
  /* estado 0 */ { 0, ..., 0, ... },            // erro é absorvente
  /* estado 1 */ { 0, ..., 2, ..., 2, ... 0 },  // letra/_ -> 2
  /* estado 2 */ { 0, ..., 2, ..., 2, ... 0 } };// letra/_/dígito -> 2

int state = 1;         // estado inicial
int c;
while ((c = getchar()) != EOF) {
  state = delta[state][c];
  if (state == 0) return ERROR;    // rejeita
  if (FINAL(state)) return ACCEPT; // aceita (FINAL(state) é state==2)
}
return ERROR; // fim do input sem atingir estado final
```

Traço com a entrada `"x1"`:

Passo	c lido	delta[state][c]	novo state	ação
1	x (letra)	delta[1]['x'] = 2	2	FINAL(2) → **return ACCEPT** já

O ciclo **nunca lê o `1`**: aceita no primeiro estado final a que chega. Isto **não** é o que um *lexer* quer (ver *longest match*, abaixo): um *lexer* real guarda a posição do último estado final visto e continua a ler até não haver transição. E cada *token* novo exige mais tabelas escritas à mão: é esse trabalho que os geradores automatizam.

Pratica agora **DFA em C (Folha lab. 2, Exercício 1)**

O `src/CLexer.c` do `scanner-c` segue o padrão "ler carácter, decidir por casos" desta secção (um `switch`/`while`, sem tabela explícita). Estende-o:

- **(a)**, **(b)** — novos tokens `{`, `}`, `;` e palavras reservadas `WHILE`, `FOR`, `INT`, `FLOAT`: o mesmo padrão de `IF`/`LPAREN` já lá.
- **(c)** — aceitar `_` como letra: é a expressão regular `[_a-zA-Z]` mal transcrita para o C (falta um `case '_':` a par de `isalpha(c)`).
- **(d)** — `REAL` (`0.5`, `123.45`): um estado novo depois do `.`.
- **(e)** — ignorar comentários `/* ... */` e `// ...`.
- **(f)** — *buffer overrun* nos identificadores: testa o comprimento máximo antes de escrever no buffer.

Cuidado com a ordem `if` vs. `ID` (ver "Que regra ganha").

Soluções: Ex. 1 (a), Ex. 1 (b), Ex. 1 (c), Ex. 1 (d), Ex. 1 (e), Ex. 1 (f)

Geradores: Alex e Flex

Um **gerador de analisadores lexicais** recebe uma lista de regras "expressão regular → ação" e produz o código de um *lexer* (faz Thompson, subconjuntos e minimização sozinho):



O **Alex** gera Haskell; o **Flex** gera C. Os dois ficheiros têm a mesma forma: código da linguagem anfitriã, **definições** com nome (`$digit/digit`), e as **regras** `padrão { ação }`, por ordem.

Exemplo resolvido **Alex**

```
{
module Lexer where
}
%wrapper "basic"
$alpha = [_a-zA-Z]
$digit = [0-9]

tokens :-
$white+                      ;  -- ignorar carateres brancos
if                           { \_ -> IF }
$alpha($alpha|$digit)*       { \s -> ID s }
$digit+                      { \s -> NUM (read s) }
$digit+"."$digit+            { \s -> REAL (read s) }

{
data Token = IF | ID String | NUM Int | REAL Double
}
```

O que está entre `{ }` é Haskell. Cada ação é uma função `String -> Token` que recebe o texto reconhecido; `;` em vez de ação quer dizer "ignorar". O `Lexer.hs` exporta `alexScanTokens :: String -> [Token]`:

```
module Main where
import Lexer
main = do
  txt <- getContents
  print (alexScanTokens txt)
```

Com `%wrapper "posn"` a ação recebe também a **posição** (`AlexPosn -> String -> Token`, com `AlexPn` posição, linha, coluna).

Exemplo resolvido **Flex**

```
%{
#include "tokens.h"
%}
%option noyywrap
alpha       [_a-zA-Z]
digit       [0-9]
%%

[ \t\n\r]+                /* skip whitespace */
if                         { return IF; }
{alpha}({alpha}|{digit})*  { return ID; }
{digit}+                   { yynum = atoi(yytext); return NUM; }
{digit}+"."{digit}+        { yyreal = atof(yytext); return REAL; }
<<EOF>>                    { return EOF; /* end of input */ }
```

Estrutura: `%{ código C %}` e definições, `%%`, regras, e opcionalmente outro `%%` com mais código C. `yylex()` devolve o próximo *token* (um inteiro definido num *header*); o texto reconhecido está na global `yytext`, e os valores guardam-se noutras globais (`yynum`, `yyreal`):

```c
#include "tokens.h"
int main(void) {
  int token;
  while ((token = yylex()) != EOF) {
    switch (token) {
      case IF:   printf("IF ");               break;
      case ID:   printf("ID(%s) ", yytext);    break;
      case NUM:  printf("NUM(%d) ", yynum);    break;
      case REAL: printf("REAL(%f) ", yyreal);  break;
    }
  }
}
```

O Flex não guarda posições: `%option yylineno` mantém a linha em `yylineno`, mas é o programador que a copia para cada *token* †.

	Alex	Flex
Linguagem gerada	Haskell (`.hs`)	C (`lex.yy.c`)
Ficheiro de entrada	`.x`	`.l`
Texto do token	argumento da ação (`\s -> ...`)	global `yytext`
Posição do token	`%wrapper "posn"` (automático)	`%option yylineno` (manual)
Arranque	`alexScanTokens :: String -> [Token]`	`yylex()` em ciclo

Que regra ganha: *longest match* e *first match*

Várias regras podem casar com o início da entrada (`if` casa com a regra `if` **e** com a dos identificadores). O Alex e o Flex decidem assim:

> **Definição** **Regras de desambiguação**
>
> 1. **Longest match**: ganha a regra que consome **mais carateres**;
> 2. **First match**: em caso de empate, ganha a que aparece **primeiro** no ficheiro.
>
> Depois do *token*, recomeça-se no carácter a seguir.

Procedimento para saber que *tokens* saem: (1) para cada regra, qual é o prefixo **mais comprido** da entrada que ela aceita? (2) fica a maior; (3) empate → a primeira no ficheiro; (4) recomeça depois desse prefixo.

Erro típico: pôr a regra dos identificadores **antes** das palavras reservadas. Com a ordem trocada, `if` empata e ganha `ID`: nunca sai `IF`. Por isso **as palavras reservadas vêm antes dos identificadores**.

Exemplo resolvido

Regras do Flex acima, entrada `if iffy 12.5`:

- **`if`**: regra `if` → 2 carateres; `ID` → 2 (`if`, o espaço pára-o); `NUM`, `REAL` → nada. Empate a 2: ganha a primeira, **`IF`**.
- **espaço**: só a regra dos brancos casa: ignora-se.
- **`iffy`**: regra `if` → 2 (`if`); `ID` → 4 (`iffy`). Ganha a mais comprida: **`ID`** com `yytext = "iffy"`.
- **espaço**: ignora-se.
- **`12.5`**: `NUM` → 2 (`12`); `REAL` → 4 (`12.5`). Ganha **`REAL`**, `yyreal = 12.5`.

Saída: `IF ID(iffy) REAL(12.500000).`

Pratica agora **Flex (Folha lab. 2, Exercício 3)**

- Reimplementa o `scanner-c` num ficheiro `Lexer.l`, com a estrutura de três partes do exemplo do Flex e as expressões regulares que escreveste no Exercício 2. Compila com `flex Lexer.l` e `gcc lex.yy.c -o Lexer`.
- Põe as palavras reservadas **antes** da regra de `ID` (*first match*) e confirma que `iffy` sai como `ID` (*longest match*).

Soluções: Ex. 3

4 Aula 4
Análise sintática e gramáticas independentes de contexto

Gramáticas independentes de contexto

A **análise sintática** verifica se a sequência de *tokens* tem a forma de um programa (parêntesis casados, operadores com os operandos certos, instruções separadas por `;`) e constrói a **árvore sintática**. Se o programa faz sentido (tipos, variáveis declaradas) já é a **análise semântica**. A forma da linguagem descreve-se com uma **gramática**.

> **Definição** **Gramática independente de contexto (CFG)**
>
> Um quádruplo $G = (\Sigma, N, S, P)$:
>
> - Σ — **terminais** (os *tokens*);
> - N — **não-terminais** (categorias: "expressão", "instrução");
> - $S \in N$ — símbolo **inicial**;
> - P — **produções** $X \to \alpha$, com $X \in N$ e α uma sequência (possivelmente vazia, ε) de terminais e não-terminais. $X \to \alpha \mid \beta$ abrevia duas produções.
>
> "Independente de contexto": o lado esquerdo é **um só não-terminal**, por isso uma produção aplica-se sem olhar ao que está à volta.

Gramática usada nesta aula (a dos slides), com $\Sigma = \{a, b\}$, $N = \{S, B\}$ e inicial S:

Nº	Produção
(1)	$S \to aSB$
(2)	$S \to \varepsilon$
(3)	$S \to B$
(4)	$B \to Bb$
(5)	$B \to b$

Derivações

> **Definição** **Derivação**
>
> Um passo $\Rightarrow$ substitui **um** não-terminal pelo lado direito de uma das suas produções; $\Rightarrow^*$ é zero ou mais passos. Quando há vários não-terminais, a derivação é †:
>
> - **mais à esquerda** (*leftmost*) se substitui sempre o mais à esquerda;
> - **mais à direita** (*rightmost*) se substitui sempre o mais à direita.
>
> As duas dão a **mesma árvore**. A análise *top-down* (LL, Aula 7) constrói a derivação mais à esquerda; a *bottom-up* (LR, Aula 5) constrói a mais à direita, de trás para a frente.

Erro típico: substituir dois não-terminais no mesmo passo, ou numa derivação mais à direita deixar um não-terminal da direita por resolver.

Exemplo resolvido

Derivação **mais à esquerda** de $aabbb$ (número da produção por cima):

$$S \overset{1}{\Rightarrow} aSB \overset{1}{\Rightarrow} aaSBB \overset{2}{\Rightarrow} aaBB \overset{4}{\Rightarrow} aaBbB \overset{5}{\Rightarrow} aabbB \overset{5}{\Rightarrow} aabbb$$

A 1 duas vezes dá os dois `a` e dois B; a 2 apaga o S; a 4 faz do primeiro B um Bb; a 5 resolve esse B em `b`, e a 5 outra vez o último B.
Derivação **mais à direita** da mesma árvore (sublinhado: o próximo a substituir):

$$S \overset{1}{\Rightarrow} aS\underline{B} \overset{5}{\Rightarrow} a\underline{S}b \overset{1}{\Rightarrow} aaS\underline{B}b \overset{4}{\Rightarrow} aaS\underline{B}bb \overset{5}{\Rightarrow} aa\underline{S}bbb \overset{2}{\Rightarrow} aabbb$$

Os B resolvem-se antes do S do meio porque estão à direita dele.

Linguagem de uma gramática e como escrever uma

Definição **L(G)**

$L(G) = \{\, w \in \Sigma^* \,:\, S \Rightarrow^* w \,\}$: as palavras **só de terminais** que se derivam do símbolo inicial.

Procedimento para descobrir $L(G)$ †:

1. **O que gera cada não-terminal sozinho?** Começa pelos que não dependem de outros.
2. **Em que ordem ficam os terminais?** (Os `a` todos antes dos `b`?)
3. **Que relação há entre as quantidades?** Conta quantos de cada saem por cada produção.
4. **Há produções que não acrescentam palavras?** (Só dão outra derivação para a mesma palavra.)
5. **Verifica** com a palavra mais pequena, com uma maior e com uma que não deve pertencer.

Exemplo resolvido **Exercício 1(a) (Folha lab. 3, também nos slides)**

Que linguagem descreve a gramática acima?

1. B **sozinho:** $B \to b$ dá `b` e $B \to Bb$ junta mais um. B gera b^+ (um ou mais `b`; não há $B \to \varepsilon$).
2. **Ordem:** $S \to aSB$ aplicada n vezes dá $a^n\, S\, B_1 \cdots B_n$: os `a` ficam todos à esquerda.
3. **Quantidades:** cada B_i dá b^{k_i} com $k_i \geq 1$. O S do meio acaba por (2) $S \to \varepsilon$, e o total de `b` é $m = k_1 + \cdots + k_n \geq n$; qualquer $m \geq n$ se atinge, aumentando um k_i. Com $n = 0$ dá a palavra vazia.
4. **(3)** $S \to B$ junta mais um B ($m \geq n + 1$): palavras que **já** saíam pela via (2). Não acrescenta palavras, só **derivações alternativas** (é o que a torna ambígua, ver abaixo).
5. **Verificação:** ε (com $n = 0$), `abb` ($n = 1$, $m = 2$) estão; `aab` não ($m < n$), `ba` não (`b` antes de `a`).

$$L(G) = \{\, a^n b^m \,:\, n \geq 0,\ m \geq n \,\}$$

Numa frase: **as seguidos de `b`s, com pelo menos tantos `b` como `a`**.

Escrever uma gramática para uma linguagem

Não há algoritmo geral, mas uma **expressão regular** traduz-se mecanicamente †, com um não-terminal novo para cada $*$, `+` ou `|`:

Expressão regular	Produções
rs (concatenação)	um lado direito com r seguido de s
$r \mid s$ (alternativa)	$X \to r$ e $X \to s$
r^* (zero ou mais)	$X \to r\,X \mid \varepsilon$
r^+ (uma ou mais)	$X \to r\,X \mid r$

Por isso **toda a linguagem regular é independente de contexto**. O contrário é falso: $\{a^n b^m \,:\, m \geq n\}$ não é regular (um autómato finito não consegue contar os `a`). Para uma linguagem que **não** é regular, pergunta **como**

se parte uma palavra em pedaços que são outra vez palavras da linguagem (ex.: o que está dentro e o que está depois de um par de parêntesis): cada pedaço é uma chamada recursiva.

Exemplo resolvido **Exercício 1(d) (Folha lab. 3)**

Gramática para `((ab*a)|(ba*b))`.

1. **Alternativa de topo**: `ab*a` ou `ba*b`, duas produções para S.
2. **`ab*a`**: `b*` precisa de um não-terminal, $B \to bB \mid \varepsilon$; a concatenação fica $S \to aBa$.
3. **`ba*b`**: da mesma forma, $A \to aA \mid \varepsilon$ e $S \to bAb$.

$$S \to aBa \mid bAb \qquad B \to bB \mid \varepsilon \qquad A \to aA \mid \varepsilon$$

Verificação: `abba` é aceite, $S \Rightarrow aBa \Rightarrow abBa \Rightarrow abbBa \Rightarrow abba$. `ab` não é: começa por `a`, só pode vir de $S \to aBa$, que acaba em `a`.

Pratica agora **Escrever gramáticas (Folha lab. 3, Exercício 1)**

Ex. 1(b) Gramática para as palavras com o mesmo número de `a`s e `b`s, por qualquer ordem. Aqui não há expressão regular (a linguagem não é regular). Pista: olha para a **primeira** letra da palavra e para o ponto onde as contagens voltam a ficar iguais pela primeira vez.
Ex. 1(c) Gramática para os parêntesis casados (`()`, `(())`, `()()`, …). Pista: uma palavra não vazia começa por um `(` que fecha num certo `)`; o que está lá dentro e o que vem a seguir são outra vez parêntesis casados.
Ex. 1(e) Gramática para `((0|1)+"."(0|1)*)|((0|1)*"."(0|1)+)`. Usa a tabela acima, como no exemplo da 1(d). Um não-terminal para "um dígito" (`0|1`) simplifica tudo.
A 1(a) e a 1(d) estão resolvidas acima.
Soluções: Ex. 1 (b), Ex. 1 (c), Ex. 1 (e)

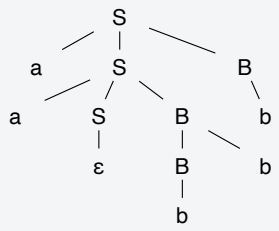
Árvores sintáticas e ambiguidade

Definição **Árvore sintática**

Cada produção $X \to \alpha_1 \ldots \alpha_n$ usada numa derivação é um nó X com n filhos, um por símbolo α_i, da esquerda para a direita ($X \to \varepsilon$ dá um nó sem filhos). As folhas, lidas da esquerda para a direita, são a palavra. Esta árvore, com um nó por símbolo, é a **árvore sintática concreta**; o compilador guarda uma versão compacta, a **AST** (Aula 6).

Exemplo resolvido

Árvore da derivação de *aabbb* acima:



A raiz usa (1): filhos `a`, o S do meio e um B (o exterior). O S do meio usa (1): `a`, o S interno e um segundo B. O S interno usa (2) e fica vazio. O segundo B usa (4), com filhos B (que usa (5)) e `b`. O B exterior usa (5). Folhas: `a`, `a`, `b`, `b`, `b`.

Definição **Gramática ambígua**

Uma gramática é **ambígua** se alguma palavra tem **duas ou mais árvores diferentes**. O compilador usa a árvore para dar **significado** ao programa (em `1+2*3`, o que se calcula primeiro), por isso as gramáticas das linguagens de programação não podem ser ambíguas, ou têm regras explícitas de desambiguação.

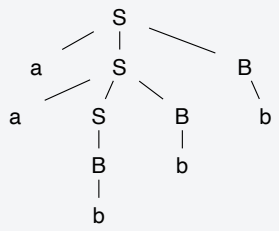
Procedimento para mostrar que é ambígua:

1. **Procura a fonte:** uma produção $X \to X\,op\,X$ (o mesmo não-terminal dos dois lados de um operador), ou dois caminhos que geram o mesmo (aqui, $S \to \varepsilon$ seguido de B, ou $S \to B$).
2. **Escolhe a palavra mais curta** onde as duas escolhas se cruzam (`1+2+3` para $E \to E + E$).
3. **Desenha as duas árvores** e confirma que as folhas dão a mesma palavra.

Erro típico: apresentar duas **derivações** diferentes como prova. Não chega: a mais à esquerda e a mais à direita são diferentes e dão a **mesma** árvore. Têm de ser duas **árvores**.

Exemplo resolvido

A gramática acima é ambígua. Fonte: (3) $S \to B$ faz o mesmo que deixar o S vazio e juntar um B. `aabbb` também se obtém com (3) no S interno: $S \overset{1}{\Rightarrow} aSB \overset{1}{\Rightarrow} aaSBB \overset{3}{\Rightarrow} aaBBB \overset{5}{\Rightarrow} aabBB \overset{5}{\Rightarrow} aabbB \overset{5}{\Rightarrow} aabbb$. Ficam três B, todos por (5), sem usar (4):



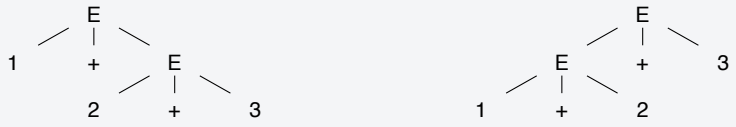
O S interno tem agora um filho B (antes não tinha filhos): árvore diferente, mesma palavra. **É ambígua.**

Eliminar ambiguidade: associatividade e precedência

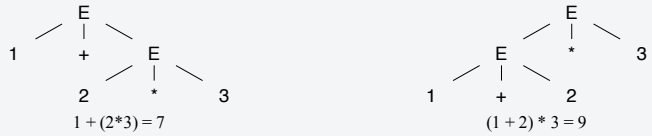
O caso central são as expressões: $E \to E + E \mid E * E \mid$ `num` $\mid (E)$ é ambígua.

Exemplo resolvido

`1+2+3` tem duas árvores (o valor é o mesmo, 6):



Em `1+2*3` as duas árvores dão **valores diferentes**, 7 ou 9:



Há duas coisas a fixar: a **associatividade** (`1+2+3` é (1+2)+3 ou 1+(2+3)?) e a **precedência** (`1+2*3` é $1 + (2\times3)$ ou $(1+2)\times3$?). Fixam-se **reescrevendo a gramática**, sem mudar a linguagem, ou declarando-as no gerador de analisadores (Aula 6).

Procedimento para reescrever †:

1. **Um não-terminal por nível de precedência**, do mais fraco (o inicial) para o mais forte; o último nível tem os átomos e os parêntesis, que voltam ao topo: $F \to$ `num` | (E).
2. **Cada nível usa o seguinte**: $E \to \cdots \mid T$, $T \to \cdots \mid F$.
3. **Associatividade à esquerda** → recursão à esquerda, $E \to E + T$; **à direita** → recursão à direita, $E \to T + E$.

Definição **Gramática de expressões desambiguada**

$$E \to E + T \mid T \qquad T \to T * F \mid F \qquad F \to \texttt{num} \mid (E)$$

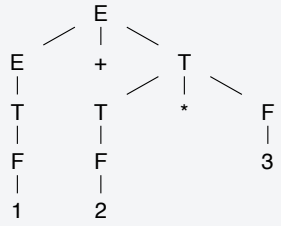
Uma **expressão** (E) é uma soma de **termos** (T), um termo é um produto de **fatores** (F), e um fator é um número ou uma expressão entre parêntesis.

- A recursão à esquerda ($E \to E + T$) dá **associatividade à esquerda**: em `1+2+3`, só o lado esquerdo pode conter outro `+`.
- Um $*$ só aparece **dentro** de um T, por isso agarra os operandos antes de a soma os usar: **$*$ tem mais precedência que `+`**.

Erros típicos: escrever $E \to E + E \mid T$ (continua ambígua: os dois lados podem ter `+`); escrever $E \to T + E$ por engano (passa a associar à direita, e `1–2–3` daria $1 - (2 - 3) = 2$).

Exemplo resolvido

Com esta gramática, `1+2*3` só tem uma árvore:



A raiz tem de ser $E \to E + T$ (o `+` só aparece aí). À esquerda, $E \to T \to F \to 1$; à direita, um T com $T \to T * F$, $T \to F \to 2$ e $F \to 3$. O $*$ fica no ramo direito do `+`: o valor é sempre $1 + (2 \times 3) = 7$.

Pratica agora **Ambiguidade em programas sequenciais (Folha lab. 3, Exercício 2)**

Gramática de "programas sequenciais" (também um exercício dos slides): $S \to S; S \mid$ `ident` $= E \mid$ `ident++` e $E \to$ `ident` | `num` | $E + E$.

Ex. 2(a) Mostra que é ambígua, com duas árvores diferentes para a mesma frase. O enunciado pergunta se há mais do que um exemplo: há, em **dois** sítios distintos. Um é nas expressões (`E→E+E`, o mesmo problema do `E→E+E` acima). O outro é nas próprias instruções: `S→S;S` tem a mesma forma que `E→E+E`, por isso tem o mesmo problema de associatividade.

Ex. 2(b) Reescreve a gramática sem ambiguidade. Usa a recursão à esquerda de `E→E+T|T` acima, aplicada nos dois sítios (também a `S`).

Soluções: Ex. 2 (a), Ex. 2 (b)

O problema do "dangling else"

$$S \to \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{if}\ E\ \texttt{then}\ S \mid \ldots$$

Com o `else` opcional, `if e1 then if e2 then s1 else s2` tem duas árvores: o `else` liga-se ao `if e2` (o mais próximo) ou ao `if e1`. Quase todas as linguagens querem **o `else` ligado ao `if` mais próximo**. Há duas formas de o impor:

- **Corrigir a gramática**, com M (*matched*: todo o `if` tem `else`) e U (*unmatched*: há um `if` por fechar):

  $$S \to M \mid U \qquad M \to \texttt{if}\ E\ \texttt{then}\ M\ \texttt{else}\ M \mid \ldots \qquad U \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ M\ \texttt{else}\ U$$

  Antes de um `else` só pode estar um M, ou seja, um `if` já fechado: o `else` fica obrigado a agarrar o `if` mais próximo.
- **Deixar a gramática** e resolver no analisador: um analisador LR escolhe *shift* no conflito (Aula 5, "Conflitos"). É o mais comum.

5 Aula 5
Análise sintática *bottom-up* (LR)

Um analisador LR lê os *tokens* e, com uma **pilha** e uma **tabela**, decide se a entrada pertence à gramática e qual é a árvore. Esta aula mostra como funciona (shift/reduce), como se **constrói a tabela** (LR(0), depois SLR(1)) e o que são os **conflitos**.

Análise *bottom-up*: shift e reduce

Definição **Análise LR**

LR = *Left-to-right parse, Rightmost derivation*: lê a entrada da esquerda para a direita e constrói a **derivação mais à direita ao contrário**, das folhas (os *tokens*) até ao símbolo inicial. Por isso é ***bottom-up***.
Tem a **entrada** (os terminais por ler) e uma **pilha** de símbolos, no início vazia. Em cada passo faz uma de duas coisas:

- ***Shift***: tira o próximo terminal da entrada e empilha-o.
- ***Reduce*** por $X \to \gamma$: se γ está no **topo** da pilha, tira γ e empilha X.

Acrescenta-se à gramática um símbolo inicial novo S', o terminal \$ (**fim da entrada**) e a produção $S' \to S\,\$$.
Aceita-se quando sobra S e a entrada é só \$.

Em relação à LL (Aula 7), a LR reconhece **mais linguagens**, a gramática reescreve-se **mais facilmente** para ela, e as ambiguidades resolvem-se com **prioridades e associatividades**, sem mexer na gramática (Aula 6).

Exemplo resolvido **Exemplo 1 dos slides: parêntesis equilibrados**

Gramática: $S' \to S\,\$$ e $S \to (S)S \mid \varepsilon$. Entrada `()`.

Pilha	Entrada	Ação	Porquê
ε	`()$`	shift	nada no topo para reduzir
`(`	`)$`	reduce $S \to \varepsilon$	dentro dos parêntesis há um S (vazio)
`(S`	`)$`	shift	
`(S)`	`$`	reduce $S \to \varepsilon$	falta o S final de $(S)S$; é vazio
`(S)S`	`$`	reduce $S \to (S)S$	o topo é o lado direito
S	`$`	accept	sobrou S e a entrada acabou

Reduce $S \to \varepsilon$ tira **zero** símbolos e empilha S. Os *reduce* lidos de baixo para cima dão a derivação mais à direita: $S \Rightarrow (S)\underline{S} \Rightarrow (\underline{S}) \Rightarrow ()$.

Exemplo resolvido **Exemplo 2 dos slides: expressões simples**

Gramática: $E' \to E\,\$$ e $E \to E + \mathtt{n} \mid \mathtt{n}$. Entrada `n+n`.

Pilha	Entrada	Ação
ε	`n+n$`	shift
`n`	`+n$`	reduce $E \to$ `n`
E	`+n$`	shift
E`+`	`n$`	shift
E`+n`	`$`	reduce $E \to E +$ `n`
E	`$`	accept

Com a pilha E`+n`, **não** se reduz o `n` sozinho a E: daria $E + E$, que não é lado direito de nada. A decisão depende do que está **por baixo** do topo, não só do topo: é o que a tabela LR resume num estado.

Tabela LR e o algoritmo de *parsing*

Para não olhar para a pilha toda em cada passo, **resume-se a pilha num estado** (um número), e as ações ficam numa **tabela** indexada por (estado no topo, próximo terminal).

Definição **Tabela de parsing LR**

Linhas = **estados**; colunas = **símbolos** (terminais, depois não-terminais). Cada entrada é:

- $s\,q$ (*shift* q): consome o próximo terminal e empilha o estado q;
- $r\,k$ (*reduce* k), sendo a produção $k\ X \to \alpha_1 \dots \alpha_n$: **tira** n **estados**, olha para o estado que fica no topo, e empilha o que estiver em (topo, X) — um $g\,q$. **Não consome** a entrada;
- $g\,q$ (*go* q): só nas colunas de não-terminais, usado depois de um *reduce*;
- a (*accept*): aceita;
- **vazia**: erro de sintaxe.

Procedimento (cada passo):

1. **Estado no topo e próximo terminal?** Lê essa entrada da tabela.
2. $s\,q$**:** empilha q e avança na entrada.
3. $r\,k$**:** quantos símbolos tem o lado direito da produção k? Tira esse número de estados ($X \to \varepsilon$ tira **zero**). Com o estado que ficou no topo e a coluna do lado esquerdo X, lê $g\,q$ e empilha q. A entrada fica igual.
4. a**:** aceita. **Vazia:** erro.

Erros típicos: depois de um *reduce*, ir buscar o *goto* ao estado que **foi tirado** em vez do que **ficou** no topo; avançar na entrada num *reduce*; tirar um estado num *reduce* por ε.

Exemplo resolvido **Tabela dos slides**

Nº	Produção
(0)	$T' \to T\,\$$
(1)	$T \to R$
(2)	$T \to aTc$
(3)	$R \to \varepsilon$
(4)	$R \to bR$

	a	b	c	$	T	R
0	s3	s4	r3	r3	g1	g2
1				a		
2			r1	r1		
3	s3	s4	r3	r3	g5	g2
4		s4	r3	r3		g6
5			s7			
6			r4	r4		
7			r2	r2		

Leitura: estado 0 com a → *shift*, vai para 3. Estado 3 com c → *reduce* pela produção 3 ($R \to \varepsilon$). Estado 0 depois de um *reduce* para T → g1. Estado 1 com $ → aceita.

Exemplo resolvido **Análise de aabbbcc com a tabela acima (1/2)**

"Estados" é a **pilha de estados** (o topo é o último número); "Símbolos" é a pilha de símbolos correspondente, só para ajudar a ler.

Estados	Símbolos	Entrada	Ação
0	ε	`aabbbcc$`	(0,a) = s3: shift 3
0 3	a	`abbbcc$`	(3,a) = s3: shift 3
0 3 3	a a	`bbbcc$`	(3,b) = s4: shift 4
0 3 3 4	a a b	`bbcc$`	(4,b) = s4: shift 4
0 3 3 4 4	a a b b	`bcc$`	(4,b) = s4: shift 4
0 3 3 4 4 4	a a b b b	`cc$`	(4,c) = r3
0 3 3 4 4 4 6	a a b b b R	`cc$`	(6,c) = r4
0 3 3 4 4 6	a a b b R	`cc$`	(6,c) = r4

- **(4,c) = r3**, $R \to \varepsilon$: tira **0** estados. Topo 4; (4,R) = g6, empilha 6.
- **(6,c) = r4**, $R \to bR$: tira **2** estados, fica `0 3 3 4 4`. Topo 4; (4,R) = g6, empilha 6.
- **(6,c) = r4** outra vez: tira 2, fica `0 3 3 4`; (4,R) = g6, empilha 6.

Exemplo resolvido **Análise de aabbbcc com a tabela acima (2/2)**

Estados	Símbolos	Entrada	Ação
0 3 3 4 6	a a b R	`cc$`	(6,c) = r4
0 3 3 2	a a R	`cc$`	(2,c) = r1
0 3 3 5	a a T	`cc$`	(5,c) = s7: shift 7
0 3 3 5 7	a a T c	`c$`	(7,c) = r2
0 3 5	a T	`c$`	(5,c) = s7: shift 7
0 3 5 7	a T c	`$`	(7,$) = r2
0 1	T	`$`	(1,$) = a: **accept**

- **(6,c) = r4**: tira 2 de `0 3 3 4 6`, fica `0 3 3`. (3,R) = g2, empilha 2.
- **(2,c) = r1**, $T \to R$: tira 1, fica `0 3 3`. (3,T) = g5, empilha 5.
- **(7,c) = r2**, $T \to aTc$: tira **3** de `0 3 3 5 7`, fica `0 3`. (3,T) = g5, empilha 5.
- **(7,$) = r2**: tira 3 de `0 3 5 7`, fica `0`. (0,T) = g1, empilha 1. Com $, (1,$) = a: aceita.

Definição **Algoritmo de parsing LR**

```
stack = empty; push(0, stack); next = getToken()
loop
  case table[top(stack), next] of
    shift s:  push(s, stack); next = getToken()
    reduce p: let X = lado esquerdo da producao p
                  n = comprimento do lado direito da producao p
              pop n elementos da pilha
              lookup table[top(stack), X] e encontra "go s"
              push(s, stack)
    accept:   termina com sucesso
    vazio:    reporta erro
```

A decisão vem **só da tabela**, e basta guardar estados na pilha. **O difícil é construir a tabela**, o que se faz uma vez, a partir da gramática.

LR(0), LR(1), LR(k) é o número de símbolos de *look-ahead* que a tabela usa para decidir. LR(0) não usa nenhum e reconhece poucas linguagens; LR(k) com $k \geq 2$ dá tabelas enormes; **LR(1) chega para quase todas as linguagens de programação**.

Autómato e tabela LR(0)

Items LR(0)

Definição **Item LR(0)**

Uma produção com uma **posição marcada** (•) no lado direito. $A \to \beta \bullet \gamma$ quer dizer: β **já está no topo da pilha**, e falta reconhecer γ.

- $A \to \bullet\gamma$ — **item inicial**: ainda nada reconhecido.
- $A \to \gamma\bullet$ — **item completo**: pode reduzir-se a A.

Um lado direito de comprimento n dá $n + 1$ items ($S \to \varepsilon$ dá só $S \to \bullet$). Os **estados** do autómato LR são **conjuntos de items**.

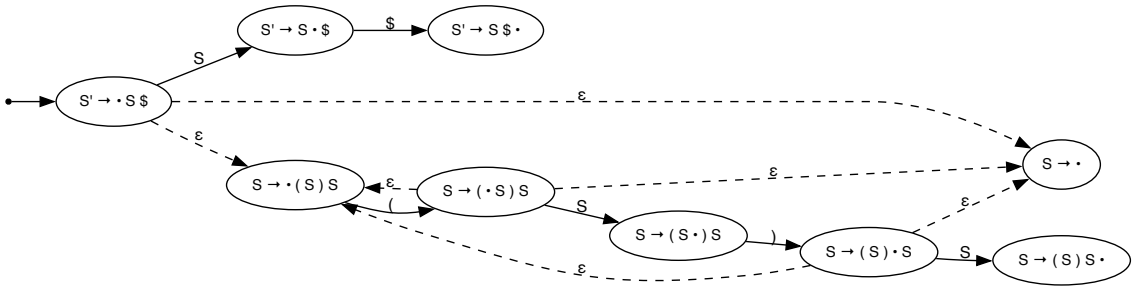
Exemplo resolvido

Os 9 items da gramática dos parêntesis:

Produções	Items
$S' \to S\,\$$	$S' \to \bullet S\,\$$ $S' \to S \bullet \$$ $S' \to S\,\$\bullet$
$S \to (S)S$	$S \to \bullet(S)S$ $S \to (\bullet S)S$ $S \to (S\bullet)S$
	$S \to (S) \bullet S$ $S \to (S)S\bullet$
$S \to \varepsilon$	$S \to \bullet$

O autómato: fecho e goto

Os items ligam-se num **NFA**: por um símbolo X, de $A \to \alpha \bullet X\gamma$ para $A \to \alpha X \bullet \gamma$ (um terminal depois de um *shift*, um não-terminal depois de um *reduce*); e por ε, de $A \to \alpha \bullet B\gamma$ para **todos** os $B \to \bullet\beta$ (para passar por cima de B é preciso começar a reconhecer um B). O estado inicial é $S' \to \bullet S\,\$$, e não há estados finais (aceita-se no \$).



NFA de items dos parêntesis; tracejado: transições-ε.

Este NFA passa a DFA pela construção de subconjuntos (Aula 2). Na prática faz-se logo o DFA, sem desenhar o NFA:

Definição **Fecho e goto**

- **fecho(I)** (o closure): para cada item de I com o ponto **antes de um não-terminal** B, junta **todos** os $B \to \bullet\beta$; repete com os que entraram, até não entrar nada.
- **goto(I, X)**: pega nos items de I com o ponto **antes de** X, avança o ponto por cima de X, e faz o **fecho**.

Procedimento para o autómato e a tabela LR(0):

1. **Aumenta e numera:** junta $S' \to S\,\$$ (produção 0) e numera as outras.
2. **Estado 0** = fecho($\{S' \to \bullet S\,\$\}$).
3. **Para cada estado: que símbolos aparecem logo a seguir a um •?** Para cada um, calcula o goto.
4. **O resultado (já com o fecho) é um estado que já existe?** Liga-o. Se não, é um estado novo. Repete até não aparecerem novos.
5. **Tabela:** transição por terminal t de i para $j \to s\,j$ em (i, t); por não-terminal $X \to g\,j$ em (i, X); item completo $A \to \gamma\bullet$ (produção k) no estado $i \to r\,k$ em **todas** as colunas de terminais da linha i; $S' \to S \bullet \$ \to$ accept em $(i, \$)$.
6. **Há entradas com duas ações?** Se não, a gramática é **LR(0)**.

Erros típicos: fazer o fecho só um nível (com $E \to (\bullet L)$ entram $L \to \bullet L, E$ e $L \to \bullet E$; este tem o ponto antes de E, por isso entram também $E \to \bullet(L)$ e $E \to \bullet\mathtt{a}$); comparar estados **antes** do fecho (dois goto diferentes podem dar o mesmo estado); pôr *reduce* nas colunas dos não-terminais.

Exemplo resolvido **DFA LR(0) dos parêntesis, passo a passo (1/2)**

Gramática: (0) $S' \to S\,\$$, (1) $S \to (S)S$, (2) $S \to \varepsilon$. Numeração dos estados como nos slides.
Estado 0 = fecho($\{S' \to \bullet S\,\$\}$). O ponto está antes de S: entram $S \to \bullet(S)S$ e $S \to \bullet$. Nestes, o ponto está antes de (ou no fim: não entra mais nada.

$$0 = \{\, S' \to \bullet S\,\$,\;\; S \to \bullet(S)S,\;\; S \to \bullet \,\}$$

De 0 (a seguir a um ponto aparecem (e S):

- goto(0, (): de $S \to \bullet(S)S$ vem $S \to (\bullet S)S$; o fecho junta $S \to \bullet(S)S$ e $S \to \bullet$. **Estado 2** = $\{S \to (\bullet S)S,\ S \to \bullet(S)S,\ S \to \bullet\}$.
- goto(0, S): de $S' \to \bullet S\,\$$ vem $S' \to S \bullet \$$; ponto antes de \$, o fecho não junta nada. **Estado 1** = $\{S' \to S \bullet \$\}$.

De 1: só por \$, que é o *accept*.

Exemplo resolvido **DFA LR(0) dos parêntesis, passo a passo (2/2)**

De 2 = $\{S \to (\bullet S)S,\ S \to \bullet(S)S,\ S \to \bullet\}$:

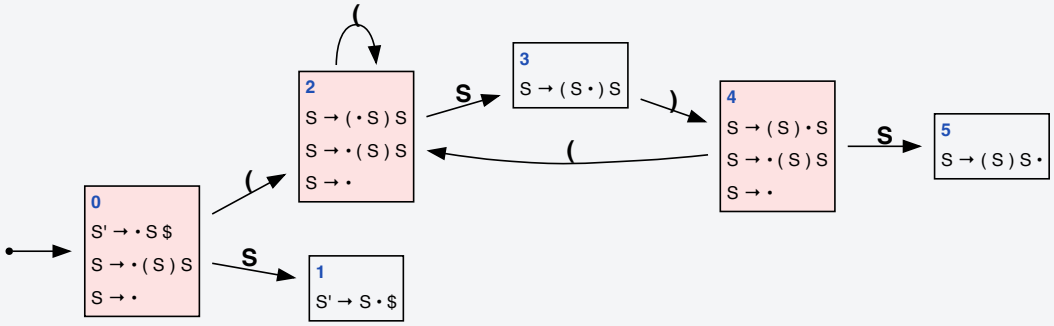
- goto(2, (): de $S \to \bullet(S)S$ vem $S \to (\bullet S)S$, cujo fecho é o **estado 2** outra vez (ciclo).
- goto(2, S): de $S \to (\bullet S)S$ vem $S \to (S\bullet)S$; ponto antes de). **Estado 3** = $\{S \to (S\bullet)S\}$.

De 3: goto(3,)) dá $S \to (S) \bullet S$; ponto antes de S, entram $S \to \bullet(S)S$ e $S \to \bullet$. **Estado 4** = $\{S \to (S) \bullet S,\ S \to \bullet(S)S,\ S \to \bullet\}$.

De 4:

- goto(4, (): de $S \to \bullet(S)S$ vem o **estado 2**.
- goto(4, S): de $S \to (S) \bullet S$ vem $S \to (S)S\bullet$. **Estado 5** = $\{S \to (S)S\bullet\}$.

De 5: nada (só um item completo). Fim: estados 0 a 5.



A rosa: os estados que vão ter conflitos na tabela LR(0).

Exemplo resolvido **Tabela LR(0) dos parêntesis**

Passo 5 do procedimento:

- *Shifts*: s2 em (0,(), (2,(), (4,(); s4 em (3,)).
- *Gotos*: g1, g3, g5 na coluna S das linhas 0, 2, 4.
- *Reduces*: 0, 2 e 4 têm $S \to \bullet$ (produção 2): r2 em todas as colunas terminais. O 5 tem $S \to (S)S\bullet$ (produção 1): r1 em todas.
- *Accept*: (1, $).

	(	)	$	S
0	**s2, r2**	r2	r2	g1
1			a	
2	**s2, r2**	r2	r2	g3
3		s4		
4	**s2, r2**	r2	r2	g5
5	r1	r1	r1	

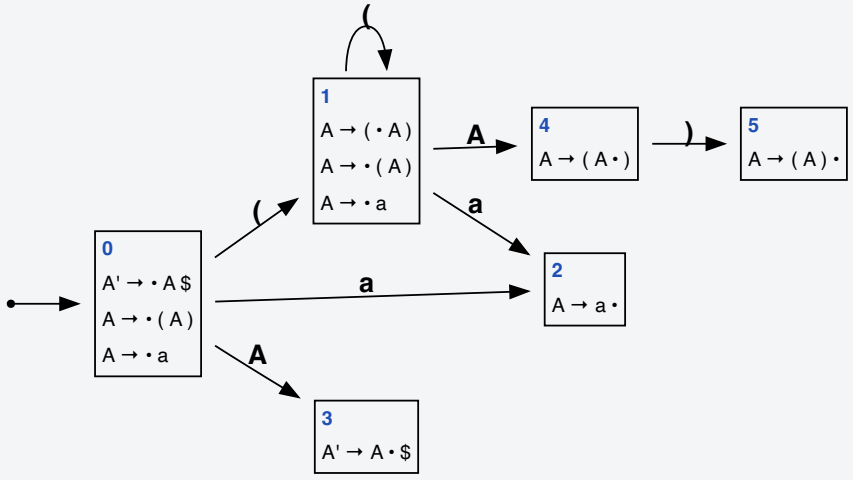
Em 0, 2 e 4 com (há **duas ações**: *shift* (abrir um par novo) ou *reduce* $S \to \varepsilon$. São **conflitos shift/reduce**: a gramática **não é LR(0)**. (A SLR(1), abaixo, resolve-os.)

Exemplo resolvido **Exercício 3(a) (Folha lab. 3) = Exercício 1 dos slides: autómato**

Mostrar que $A \to (A) \mid \mathtt{a}$ é LR(0). Passo 1: (0) $A' \to A\,\$$, (1) $A \to (A)$, (2) $A \to \mathtt{a}$.
Estado 0 = fecho($\{A' \to \bullet A\,\$\}$): ponto antes de A, entram $A \to \bullet(A)$ e $A \to \bullet\mathtt{a}$. $0 = \{A' \to \bullet A\,\$,\ A \to \bullet(A),\ A \to \bullet\mathtt{a}\}$.

- goto(0, () = fecho($\{A \to (\bullet A)\}$) = $\{A \to (\bullet A),\ A \to \bullet(A),\ A \to \bullet\mathtt{a}\}$: **estado 1**.
- goto(0, a) = $\{A \to \mathtt{a}\bullet\}$: **estado 2**.
- goto(0, A) = $\{A' \to A \bullet \$\}$: **estado 3**.

De 1: goto(1, () = **estado 1**; goto(1, a) = **estado 2**; goto(1, A) = $\{A \to (A\bullet)\}$: **estado 4**.
De 2 (só item completo) e **de 3** (só $, accept): nada.
De 4: goto(4,)) = $\{A \to (A)\bullet\}$: **estado 5**, sem transições.



Exemplo resolvido **Exercício 3(a) (Folha lab. 3): tabela e verificação**

Shifts: s1 em (0,`(`) e (1,`(`); s2 em (0,`a`) e (1,`a`); s5 em (4,`)`). *Gotos*: g3 em (0,*A*), g4 em (1,*A*). *Reduces*: estado 2 (produção 2) r2 em todas as colunas terminais; estado 5 (produção 1) r1 em todas. *Accept*: (3,$).

	(	)	a	$	A
0	s1		s2		g3
1	s1		s2		g4
2	r2	r2	r2	r2	
3				a	
4		s5			
5	r1	r1	r1	r1	

Nenhuma entrada tem duas ações (os estados com *reduce* não têm *shift*): a gramática **é LR(0)**. Teste com `((a))`:

Estados	Símbolos	Entrada	Ação
0	ε	`((a))$`	s1
0 1	(	`(a))$`	s1
0 1 1	((	`a))$`	s2
0 1 1 2	((a	`))$`	r2: tira 1, topo 1, (1,A) = g4
0 1 1 4	((A	`))$`	s5
0 1 1 4 5	((A)	`)$`	r1: tira 3, topo 1, (1,A) = g4
0 1 4	(A	`)$`	s5
0 1 4 5	(A)	`$`	r1: tira 3, topo 0, (0,A) = g3
0 3	A	`$`	accept

Pratica agora **Autómato e tabela LR(0) (Folha lab. 3, Exercícios 4 e 5)**

Ex. 4 Gramática $E \to (L) \mid \texttt{a}$, $L \to L, E \mid E$.

- **(a)** Derivação de `((a),a,(a,a))`. Usa as derivações da Aula 4; escolhe sempre o não-terminal mais à esquerda para não te perderes.
- **(b)** Autómato LR(0). Atenção ao estado depois de `(`: tem o ponto antes de L, e $L \to \bullet L, E$ também tem o ponto antes de L.
- **(c)** É LR(0)? Procura estados com um item completo **e** outro item (com *shift* ou outro *reduce*).

Ex. 5 Declarações de C: $Decl \to Type\ Varlist$ `;`, $Type \to$ `int` | `float`, $Varlist \to Varlist$ `,` `ident` | `ident`.

- **(a)** Autómato e tabela LR(0), como no Exercício 3(a) acima.
- **(b)** Simula `int x,y,z;` com a pilha de estados, como no teste de `((a))` acima (o analisador lexical dá `int ident , ident , ident ;`).

Soluções: Ex. 4 (a), Ex. 4 (b), Ex. 4 (c), Ex. 5 (a), Ex. 5 (b)

Análise SLR(1)

A LR(0) põe o *reduce* em todas as colunas, e por isso tem conflitos em gramáticas simples (a dos parêntesis). A **SLR(1)** (*Simple LR*) olha para o **próximo terminal**: só reduz para A se esse terminal **pode vir a seguir a um** A. Esse conjunto é o FOLLOW(A).

NULLABLE, FIRST e FOLLOW

Servem para a SLR(1) e para a LL(1) (Aula 7).

> **Definição** **NULLABLE, FIRST e FOLLOW**
>
> Numa gramática com inicial S, e γ uma sequência de símbolos:
>
> - **NULLABLE(γ)**: verdade se $\gamma \Rightarrow^* \varepsilon$ (γ é **anulável**).
> - **FIRST(γ)** $= \{x \in \Sigma : \gamma \Rightarrow^* x\beta\}$: os terminais por que podem **começar** as palavras derivadas de γ.
> - **FOLLOW(X)** $= \{c \in \Sigma : S \Rightarrow^* \alpha X c \beta\}$: os terminais que podem aparecer **logo a seguir** a X. Com $S' \to S\,\$$, o \$ está em FOLLOW(S).

As definições não dizem como calcular; para isso há **equações** (a terminal, α, β sequências, $X \to \gamma_1 \mid \cdots \mid \gamma_n$ todas as produções de X):

	NULLABLE	FIRST
ε	True	$\varnothing$
a	False	$\{a\}$
$\alpha\beta$	NULLABLE(α) $\wedge$ NULLABLE(β)	FIRST(α) $\cup$ FIRST(β) se NULLABLE(α); senão FIRST(α)
X	NULLABLE(γ_1) $\vee \cdots \vee$ NULLABLE(γ_n)	FIRST(γ_1) $\cup \cdots \cup$ FIRST(γ_n)

FOLLOW: para **cada ocorrência** de um não-terminal X num lado direito, $Y \to \alpha X \beta$:

1. FOLLOW(X) $\supseteq$ FIRST(β);
2. se NULLABLE(β) (incluindo β vazio), FOLLOW(X) $\supseteq$ FOLLOW(Y).

Procedimento (ponto fixo):

1. **NULLABLE primeiro**, depois **FIRST**, depois **FOLLOW** (cada um usa os anteriores).
2. Começa com tudo False / $\varnothing$.
3. Calcula os lados direitos das equações com os valores atuais; repete até uma passagem não mudar nada.

Termina sempre (os conjuntos só crescem e são finitos), e dá a **menor** solução. Nos slides cada iteração usa **só a coluna anterior**; à mão pode usar-se logo cada valor novo na mesma passagem, o que chega ao mesmo resultado em menos passagens (interpretação).

Erros típicos: pôr ε dentro do FIRST (aqui o vazio é o NULLABLE, à parte); esquecer o \$ em FOLLOW(S); no FOLLOW de um X no fim de $Y \to \alpha X$, juntar FIRST(Y) em vez de **FOLLOW(Y)**; em $\alpha\beta$ com α anulável, esquecer o FIRST(β).

Exemplo resolvido **NULLABLE e FIRST das expressões aritméticas**

$E \to E + T \mid T$, $T \to T * F \mid F$, $F \to \texttt{num} \mid (E)$.
NULLABLE: $N(E) = (N(E) \wedge N(+) \wedge N(T)) \vee N(T)$, $N(T) = (N(T) \wedge N(*) \wedge N(F)) \vee N(F)$, $N(F) = N(\texttt{num}) \vee N((E)) =$ False. Partindo de tudo False, a iteração 1 dá tudo False: estabiliza logo. **Nada é anulável.**
FIRST: como E e T não são anuláveis, $\text{FIRST}(E + T) = \text{FIRST}(E)$ e $\text{FIRST}(T * F) = \text{FIRST}(T)$:

- $\text{FIRST}(E) = \text{FIRST}(E) \cup \text{FIRST}(T)$
- $\text{FIRST}(T) = \text{FIRST}(T) \cup \text{FIRST}(F)$
- $\text{FIRST}(F) = \text{FIRST}(\text{num}) \cup \text{FIRST}((E)) = \{\texttt{num}, (\}$

	0	1	2	3	4
FIRST(E)	∅	∅	∅	{num, (}	{num, (}
FIRST(T)	∅	∅	{num, (}	{num, (}	{num, (}
FIRST(F)	∅	{num, (}	{num, (}	{num, (}	{num, (}

Na 1 só muda FIRST(F), que é constante; na 2, FIRST(T) recebe o FIRST(F) da coluna 1; na 3, FIRST(E) recebe o FIRST(T) da coluna 2; a 4 é igual à 3: ponto fixo. **FIRST(E) = FIRST(T) = FIRST(F) =** $\{\texttt{num}, (\}$**.**

Exemplo resolvido **FOLLOW da gramática dos parêntesis**

Produções: $S' \to S\,\$$, $S \to (S)S$, $S \to \varepsilon$. Ocorrências de S num lado direito:

1. $S' \to \underline{S}\,\$$: FIRST($) = {$}, logo $ ∈ FOLLOW(S).
2. $S \to (\underline{S})S$: FIRST()S) = {)}, logo) ∈ FOLLOW(S).
3. $S \to (S)\underline{S}$: no fim, junta FOLLOW(S) a si próprio: nada.

Segunda passagem: nada muda. **FOLLOW(S) =** {), $ }: um S acaba antes de um) ou no fim da entrada.

Exemplo resolvido **FOLLOW das expressões aritméticas**

$E' \to E\,\$$, $E \to E + T \mid T$, $T \to T * F \mid F$, $F \to \texttt{num} \mid (E)$, com NULLABLE e FIRST do exemplo anterior.

- E em $E' \to \underline{E}\,\$$ ($), $E \to \underline{E} + T$ (+) e $F \to (\underline{E})$ ( ). FOLLOW(E) = {$, +,)}.
- T em $E \to E + \underline{T}$ e $E \to \underline{T}$ (no fim: junta FOLLOW(E)) e em $T \to \underline{T} * F$ (∗). FOLLOW(T) = {$, +,), ∗}.
- F em $T \to T * \underline{F}$ e $T \to \underline{F}$, sempre no fim: junta FOLLOW(T). FOLLOW(F) = {$, +,), ∗}.

Segunda passagem: nada muda.

Tabela SLR(1)

> **Definição** **Tabela SLR(1)**
>
> Mesmo **autómato** e mesmo **algoritmo** do LR(0). Muda só o passo dos *reduces*: um item completo $A \to \gamma\bullet$ (produção k) no estado i dá $r\,k$ **só nas colunas dos terminais de FOLLOW(A)**. A gramática é **SLR(1)** se a tabela não tiver conflitos.

Procedimento: (1) autómato LR(0); (2) FOLLOW de cada não-terminal que tenha item completo; (3) tabela com os *reduces* só no FOLLOW; (4) há entradas duplas?

Exemplo resolvido **Tabela SLR(1) dos parêntesis**

Mesmo DFA (estados 0 a 5), mesmos *shifts* e *gotos*. Os *reduces* vão só para FOLLOW(S) = {), $ }:

- 0, 2 e 4 ($S \to \bullet$, produção 2): r2 em) e $, **não** em (;
- 5 ($S \to (S)S\bullet$, produção 1): r1 em) e $.

	(	)	$	S
0	s2	r2	r2	g1
1			a	
2	s2	r2	r2	g3
3		s4		
4	s2	r2	r2	g5
5		r1	r1	

Sem conflitos: a gramática **é SLR(1)** (sem ser LR(0)). O conflito em (desaparece porque (não está em FOLLOW(S): com (à frente, um S vazio nunca estaria certo, e faz-se *shift*.

Exemplo resolvido **Análise de (()) com a tabela SLR(1)**

Estados	Símbolos	Entrada	Ação
0	ε	(())$	s2
0 2	(	())$	s2
0 2 2	((	))$	r2: tira 0, topo 2, (2,S) = g3
0 2 2 3	((S	))$	s4
0 2 2 3 4	((S)	)$	r2: tira 0, topo 4, (4,S) = g5
0 2 2 3 4 5	((S) S	)$	r1: tira 4, topo 2, (2,S) = g3
0 2 3	(S	)$	s4
0 2 3 4	(S)	$	r2: tira 0, topo 4, (4,S) = g5
0 2 3 4 5	(S) S	$	r1: tira 4, topo 0, (0,S) = g1
0 1	S	$	accept

No primeiro passo (estado 0 com (), onde a LR(0) tinha conflito, a SLR(1) diz s2.

Pratica agora **SLR(1) (Folha lab. 3, Exercício 3(b)) = Exercício 2 dos slides**

Ex. 3(b) Mostra que $T \to R$, $T \to aTc$, $R \to \varepsilon$, $R \to bR$ é SLR(1): constrói o autómato e a tabela. Justifica que **não** é LR(0) mostrando os conflitos da tabela LR(0).
Pista: é a gramática da "Tabela dos slides", por isso podes comparar no fim. Os slides avisam que a tua numeração dos estados pode sair diferente, mas sem conflitos. Calcula FOLLOW(T) e FOLLOW(R) primeiro (a produção $T \to R$ faz com que FOLLOW(T) entre em FOLLOW(R)).
Soluções: Ex. 3 (b)

Conflitos e como resolvê-los

Definição **Conflito**

Uma entrada da tabela com **duas ou mais ações**:

- **shift/reduce**: um *shift* e um *reduce* na mesma entrada;
- **reduce/reduce**: dois *reduce* na mesma entrada.

Uma gramática ambígua dá **sempre** conflitos (duas árvores querem dizer que nalgum passo duas ações levam à aceitação). O contrário não: a dos parêntesis não é ambígua e tinha conflitos LR(0). Para os eliminar:

- **reescrever a gramática** (Aula 4);
- declarar **associatividades e precedências** dos *tokens* (Aula 6);
- escolher ***shift*** em vez de *reduce*, que é o que os geradores fazem por omissão.

Para decidir um conflito à mão: escreve uma entrada que chegue a esse estado com esse terminal à frente, e vê **que árvore dá cada ação**; fica a que dá a árvore que queres.

Exemplo resolvido **Dangling else num analisador SLR(1)**

$S \to$ `if cond then` S `else` $S \mid$ `if cond then` $S \mid$ `skip`. Há um estado com estes dois items:

$$S \to \texttt{if cond then}\ S\ \bullet \qquad\qquad S \to \texttt{if cond then}\ S\ \bullet\ \texttt{else}\ S$$

Com `else` à frente pode fazer-se **shift** do `else` (segundo item) ou **reduce** pela produção sem `else` (primeiro item), porque `else` $\in$ FOLLOW(S) = {`else`, \$}. Conflito shift/reduce.
Em `if cond then if cond then skip else skip`, com a pilha `if cond then if cond then S` e `else` à frente:

- **shift**: o `else` junta-se ao `if` **de dentro**: `if cond then { if cond then skip else skip }`;
- **reduce**: fecha o `if` de dentro sem `else`, e o `else` fica para o de fora: `if cond then { if cond then skip } else skip`.

Quer-se a primeira (Pascal, C, Java: o `else` liga-se ao `if` mais próximo), por isso o *shift* por omissão resolve este caso sem mexer na gramática.

Extras: LR(1) e LALR(1)

A SLR(1) não chega para algumas construções das linguagens de programação. A **LR(1)** resolve-as, mas pode ter **muitos mais estados**; os geradores usam a **LALR(1)**, mais compacta. Percebendo a SLR(1) já se percebe e usa um gerador; o resto fica numa linha cada.

- **Item LR(1)**: $(A \to \alpha \bullet \beta,\ a)$, um item LR(0) com um terminal a de *look-ahead*: "α está no topo e o resto da entrada deriva de βa".
- **Fecho**: de $(A \to \alpha \bullet B\gamma,\ a)$ entram $(B \to \bullet\beta,\ b)$ para **todos** os $B \to \beta$ e **todos** os $b \in \text{FIRST}(\gamma a)$. Avançar o ponto não muda o *look-ahead*.
- **Reduce** $A \to \alpha$ só quando o estado tem $(A \to \alpha\bullet,\ a)$ **e o próximo terminal é** a (mais preciso do que o FOLLOW).
- **LALR(1)**: junta os estados LR(1) com os mesmos items LR(0); fica com o número de estados do LR(0) e *look-aheads* mais precisos que o FOLLOW.

Exemplo resolvido **O início do autómato LR(1) dos parêntesis †**

$S' \to S\,\$,\ S \to (S)S \mid \varepsilon$.
Estado inicial. De $S' \to \bullet S\,\$$: ponto antes de S, com $\gamma = \$$, FIRST(\$) = {\$}. Entram $(S \to \bullet(S)S,\ \$)$ e $(S \to \bullet,\ \$)$.
Depois de `(`. De $(S \to \bullet(S)S,\ \$)$ vem $(S \to (\bullet S)S,\ \$)$. Ponto antes de S, com $\gamma =)S$: FIRST()S\$) = {)}. Entram $(S \to \bullet(S)S,\ ))$ e $(S \to \bullet,\ ))$.
Aqui o *reduce* $S \to \varepsilon$ faz-se **só com `)`** (em SLR(1) era com `)` e \$).

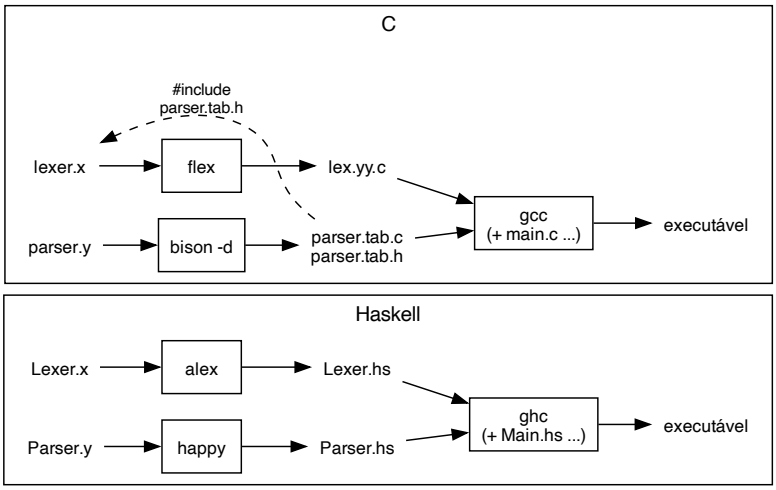
6 Aula 6
Geradores de analisadores sintáticos e sintaxe abstrata

A tabela LR só se faz à mão para gramáticas pequenas; para uma linguagem real usa-se um **gerador**. Esta aula: os geradores **Happy** (Haskell) e **Bison** (C), as **ações semânticas** que calculam valores ou constroem a árvore, os **conflitos** resolvidos com precedências, e a **sintaxe abstrata** (a AST), que é o que a análise sintática entrega.

Geradores de analisadores LR

> **Definição** **Gerador de analisadores sintáticos**
> Recebe uma **gramática** com uma ação em cada produção e produz o **código** de um analisador LR para ela: o autómato, a tabela e o ciclo de *parsing* da Aula 5. **Yacc** (o original do UNIX, para C), **Bison** (a versão GNU, C/C++) e **Happy** (Haskell) geram analisadores **LALR(1)**.

Cada um trabalha com um gerador lexical da Aula 3: **Happy + Alex** em Haskell, **Bison + Flex** em C.




```
$ alex Lexer.x                       $ bison -d parser.y
$ happy Parser.y                     $ flex lexer.x
$ ghc -o prog Lexer.hs Parser.hs ...  $ gcc -o prog parser.tab.c lex.yy.c ...
```

Os dois lados têm de concordar nos *tokens*. Em C, o `-d` do Bison gera o *header* `parser.tab.h` com os códigos dos *tokens*, que o *lexer* inclui. Em Haskell, o tipo `Token` define-se no `Lexer.x` e o `Parser.y` importa-o.

Happy

Um ficheiro `Parser.y` tem cinco partes. Exemplo dos slides, expressões com `+` e parêntesis já desambiguadas (Aula 4):

```
{                                   -- (1) código Haskell do início
module Parser where
import Lexer
}

%name parser                        -- (2) diretivas
%tokentype { Token }
%error { parseError }
```

```
%token                              -- (3) tokens
num         { TOK_NUM $$ }
'+'         { TOK_PLUS }
'('         { TOK_LPAREN }
')'         { TOK_RPAREN }

%%                                  -- (4) gramática

Exp : Term                  { () }
    | Exp '+' Term          { () }

Term : num                  { () }
     | '(' Exp ')'          { () }

{                                   -- (5) código Haskell do fim
parseError :: [Token] -> a
parseError toks = error "parse error"
}
```

com `data Token = TOK_NUM Int | TOK_PLUS | TOK_LPAREN | TOK_RPAREN` no `Lexer.x`.

- **(1)** e **(5)**: Haskell copiado para o `Parser.hs`. A `parseError` é chamada num erro sintático (entrada vazia da tabela).
- **(2)**: `%name` é o **nome da função** gerada; `%tokentype` o tipo dos *tokens*; `%error` a função de erro.
- **(3)**: `%token` liga cada **terminal** da gramática (à esquerda) ao **padrão Haskell** do *token* (à direita). O `$$` marca o **valor** do *token*: o de `TOK_NUM 42` é `42`. Sem `$$`, o valor é o próprio *token*.
- **(4)**: as produções, com `:` no lugar de →. **O primeiro não-terminal é o símbolo inicial.**

Aqui as ações devolvem todas `()`, por isso `parser :: [Token] -> ()` só **reconhece**. O programa principal faz `print (parser $ alexScanTokens txt)` sobre o `getContents`: imprime `()` se a entrada respeita a gramática, ou a `parseError` lança uma exceção.

Ações semânticas

Um **compilador** quer uma **representação do programa** (a árvore); um **interpretador** quer o **resultado**. As duas coisas fazem-se com as **ações** das produções.

> **Definição** **Ação semântica e valor semântico**
> Cada símbolo tem um **valor semântico**: o de um **terminal** vem do `%token` (o `$$`); o de um **não-terminal** é o resultado da ação da sua produção. Numa ação, **`$1`, `$2`, `$3`, …** são os valores dos símbolos do **lado direito**, contando todos, terminais incluídos: em `Exp : Exp '+' Term`, `$1` é o `Exp`, `$2` o `'+'` e `$3` o `Term`.
> **A ação corre no *reduce*** dessa produção †: os símbolos do lado direito estão no topo da pilha, cada um com o seu valor; a ação calcula o valor do lado esquerdo, que é empilhado no lugar deles. Por isso as ações correm **de baixo para cima** na árvore: os filhos antes do pai.

Erro típico: contar só os não-terminais (`Exp '+' Term` → "`$2` é o `Term`"). Conta-se **cada** símbolo: o `Term` é `$3`.

Construir a árvore. Declara-se um tipo algébrico e cada ação monta o nó a partir dos filhos:

```
Exp : Term                { $1 }
    | Exp '+' Term        { Add $1 $3 }

Term : num                { Num $1 }
     | '(' Exp ')'        { $2 }

{
data Exp = Num Int | Add Exp Exp deriving Show
}
```

`Exp : Term { $1 }` passa a árvore do termo sem nó novo; `Add $1 $3` deixa o `'+'` de fora (o construtor já diz que é soma); `'(' Exp ')' { $2 }` faz desaparecer os parêntesis (a forma da árvore guarda o agrupamento). Fica `parser :: [Token] -> Exp`, e o resultado já é a **AST** (ver abaixo).

Calcular o resultado. Com `{ $1 + $3 }` na soma, `{ $1 }` em `num` e o resto igual, `parser :: [Token] -> Int` é um **interpretador**.

Exemplo resolvido **Os valores na pilha ao analisar (1+2)+3**

Ações do interpretador; cada símbolo aparece como `símbolo:valor`. As decisões *shift/reduce* são as da tabela LALR(1) do Happy.

#	Pilha	Entrada	Ação	Valor
1		`(1+2)+3`	shift `(`	
2	`(`	`1+2)+3`	shift `num`	*token*: 1
3	`( num:1`	`+2)+3`	reduce `Term : num`	`$1` = 1
4	`( Term:1`	`+2)+3`	reduce `Exp : Term`	`$1` = 1
5	`( Exp:1`	`+2)+3`	shift `'+'`	
6	`( Exp:1 +`	`2)+3`	shift `num`	*token*: 2
7	`( Exp:1 + num:2`	`)+3`	reduce `Term : num`	`$1` = 2
8	`( Exp:1 + Term:2`	`)+3`	reduce `Exp : Exp '+' Term`	1+2 = 3
9	`( Exp:3`	`)+3`	shift `')'`	
10	`( Exp:3 )`	`+3`	reduce `Term : '(' Exp ')'`	`$2` = 3
11	`Term:3`	`+3`	reduce `Exp : Term`	`$1` = 3
12	`Exp:3`	`+3`	shift `'+'`	
13	`Exp:3 +`	`3`	shift `num`	*token*: 3
14	`Exp:3 + num:3`		reduce `Term : num`	`$1` = 3
15	`Exp:3 + Term:3`		reduce `Exp : Exp '+' Term`	3+3 = 6
16	`Exp:6`		accept	**6**

No passo 8 o *reduce* tira os três símbolos do topo: `Exp:1` é `$1`, `+` é `$2`, `Term:2` é `$3`, e a ação dá `$1 + $3`. No passo 10, `(` é `$1`, `Exp:3` é `$2` e `)` é `$3`: a ação é `$2`.
Com as ações que constroem a árvore, os *shift/reduce* são os mesmos (a tabela só depende da gramática); só mudam os valores: passos 3–4 `Num 1`; 7 `Num 2`; 8 `Add (Num 1) (Num 2)`; 10–11 o mesmo; 14 `Num 3`; 15 `Add (Add (Num 1) (Num 2)) (Num 3)`, que é o resultado.

Pratica agora **Ações semânticas (exercício inventado)**

Ex. A Com a gramática `Exp : Term | Exp '+' Term`, `Term : num | '(' Exp ')'` desta secção:

- **(a)** Faz a tabela "pilha / entrada / ação / valor" para `2+(3+4)`, com as ações do interpretador (`$1 + $3`, …). Diz em cada *reduce* quem são `$1`, `$2`, `$3`.
- **(b)** Que valor devolve `parser` para a mesma entrada com as ações que constroem a árvore (`Add $1 $3`, `Num $1`, `$2`)? Compara a forma da árvore com a de `(1+2)+3` no exemplo acima.

Soluções: Ex. A (a), Ex. A (b)

Bison

A calculadora dos slides em Bison (gerador LR do GNU para C, usado com o Flex):

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex (void);
void yyerror (char const *);
%}
%define api.value.type {int}
%token TOK_NUM

%%

top : exp            { printf("%d\n", $1); }
    ;

exp :  term          { $$ = $1; }
    | exp '+' term   { $$ = $1 + $3; }
    ;

term : TOK_NUM       { $$ = $1; }
     | '(' exp ')'   { $$ = $2; }
     ;

%%

void yyerror (char const *msg) {
  printf("parse error: %s\n", msg);
  exit(-1);
}

int main(void) {
  yyparse();
}
```

A estrutura é a do Flex (`%{ C %}` e declarações, `%%`, regras, `%%`, C). Cada regra acaba em `;`, e cada ação é um **bloco C** que dá o resultado **atribuindo a `$$`**. A regra `top` é o símbolo inicial: a sua ação corre uma vez, no último *reduce*, e imprime o resultado. O resto compara-se com o Happy na tabela:

	Happy	Bison
Linguagem gerada	Haskell (`Parser.hs`)	C (`parser.tab.c`, `.h` com `-d`)
Analisador lexical	Alex	Flex
Ação semântica	expressão Haskell	bloco C com `$$ = ...;`
Tipo dos valores	um por não-terminal (inferido)	`%define api.value.type` (um só) ou `%union`
Valor dos terminais	`$$` no padrão de `%token`	o *lexer* escreve em `yylval`
Função gerada	nome de `%name`, recebe `[Token]`	`yyparse()`, chama `yylex()`
Símbolo inicial	primeiro não-terminal	primeira regra
Erros	função de `%error`	`yyerror`
Relatório do autómato	`happy -i` → `Parser.info`	`bison --report=all` → `parser.output`

Valor de um terminal. O *lexer* põe-no na global `yylval` antes de devolver o *token*: `[0-9]+ { yylval = atoi(yytext); return TOK_NUM; }`.

Terminais de um carácter †, como `'+'` e `'('`, usam-se na gramática sem os declarar: o código do *token* é o código ASCII, e o Flex faz `return yytext[0];`. Os *tokens* com nome (`%token TOK_NUM`) recebem números a partir de 258; o fim da entrada é o *token* 0.

Conflitos, associatividade e precedência

Os geradores constroem a tabela LALR(1) e **reportam os conflitos** (Aula 5). Com a gramática ambígua da Aula 4:

```
Exp : num               { ... }
    | Exp '+' Exp       { ... }
    | Exp '*' Exp       { ... }

$ happy Parser.y
shift/reduce conflicts:  4
```

Exemplo resolvido **De onde vêm os 4 conflitos (relatório real do Happy) †**

`happy -i` escreve o relatório; os conflitos estão em dois estados (o Happy escreve `.` em vez de •):

```
State 6
    Exp -> Exp . '+' Exp        (rule 2)
    Exp -> Exp . '*' Exp        (rule 3)
    Exp -> Exp '*' Exp .        (rule 3)

    '+'     shift, and enter state 4
            (reduce using rule 3)
    '*'     shift, and enter state 5
            (reduce using rule 3)
    %eof    reduce using rule 3
```

No estado 6 a pilha acaba em `Exp * Exp`. O item completo diz "reduz"; os outros dizem que o último `Exp` pode começar um `Exp + Exp` ou `Exp * Exp` maior, e então faz-se *shift* do operador. Com `+` à frente há conflito, com `*` também: 2. O estado 7 é igual com `Exp -> Exp '+' Exp .`: mais 2. **Total: 2 estados × 2 operadores = 4.** A linha entre parêntesis é a ação que **perdeu**: o gerador **escolheu *shift***.

O *shift* por omissão **pode estar errado**. Sem declarações, `2*3+4` dá **14** †: com `Exp * Exp` na pilha e `+` à frente, faz *shift* em vez de fechar `2*3`; no fim reduz `3+4 = 7` e depois `2*7 = 14`. O *shift* sistemático põe **todos os operadores a associar à direita, com a mesma precedência**. Resolve-se reescrevendo a gramática (Aula 4) ou declarando associatividade e precedência (igual no Happy e no Bison).

Definição **Declarações de associatividade e precedência**

Antes do `%%`:

```
%nonassoc '<' '>' '=='          -- precedência mais baixa
%left '+' '-'
%left '*' '/'                   -- precedência mais elevada
```

- `%left` (associa à esquerda), `%right` (à direita), `%nonassoc` (não associa: `1<2<3` é erro).
- **A ordem das linhas dá a precedência**: cada linha tem **mais** do que as anteriores; *tokens* na mesma linha têm a mesma.

Procedimento † (o que o gerador faz num conflito entre **reduzir pela regra** *R* e **fazer *shift* do *token*** *a*):

1. **Precedência de *R*?** É a do seu **último terminal** (em `Exp : Exp '+' Exp`, a do `'+'`).
2. **Compara com a de *a*:** *a* com **mais** → *shift*; com **menos** → *reduce*.
3. **Iguais?** Decide a associatividade: `%left` → *reduce*, `%right` → *shift*, `%nonassoc` → **erro sintático**.

4. **Algum dos dois sem precedência?** Fica o *shift*, e o conflito continua a ser reportado.

Erro típico: achar que a primeira linha é a mais forte. É ao contrário: **a última linha ganha**.

Exemplo resolvido **Associatividade: 1+2+3 (um só operador)**

Depois de `1+2`, pilha `Exp + Exp`, próximo `+`:

```
Exp -> Exp . '+' Exp        (rule 2)
Exp -> Exp '+' Exp .        (rule 2)

'+'     shift, and enter state 6
        (reduce using rule 2)
```

(1) A regra 2 tem a precedência do `'+'`; (2) o *token* é o `'+'`: igual;
(2) decide a associatividade:

- `%left '+'` → ***reduce***: $(1 + 2) + 3$.
- `%right '+'` → ***shift***: $1 + (2 + 3)$.
- `%nonassoc '+'` → `1+2+3` é **erro sintático** (é o que se quer para `<`).

Exemplo resolvido **Precedência: 1+2*3 (dois operadores)**

Depois de `1+2`, próximo `*`:

```
Exp -> Exp . '+' Exp        (rule 2)
Exp -> Exp '+' Exp .        (rule 2)
Exp -> Exp . '*' Exp        (rule 3)

'*'     shift, and enter state 7
        (reduce using rule 2)
```

(1) A regra 2 tem a precedência do `'+'`; (2) o *token* é `'*'`. Com `%left '+'` **antes** de `%left '*'`, o `*` tem mais → ***shift***: $1 + (2 \times 3)$.

No caso inverso, `2*3+4`: a pilha é `Exp * Exp` (regra 3, precedência do `*`) e o *token* `+` tem menos → ***reduce***: $(2 \times 3) + 4 = 10$. Com estas duas declarações o relatório deixa de ter conflitos.

Dangling else. Nem sempre é preciso eliminar o conflito. O relatório mostra

```
Stm -> if Exp then Stm .              (rule 1)
Stm -> if Exp then Stm . else Stm     (rule 2)

else    shift, and enter state 15
        (reduce using rule 1)
```

e o *shift* por omissão liga o `else` ao `if` mais próximo, que é o correto (Aula 5): o gerador avisa, mas o analisador fica certo.

Em resumo, para cada conflito do relatório: (1) que árvore dá cada ação, e qual queres? (2) reescreve a gramática, declara `%left`/`%right`/`%nonassoc` (operadores binários) ou deixa o *shift* (só se é o que queres); (3) volta a gerar e confirma que o conflito desapareceu.

Pratica agora **Precedências no papel (exercício inventado)**

Ex. B Gramática ambígua `Exp : num | Exp '+' Exp | Exp '-' Exp | Exp '*' Exp | Exp '<' Exp`, com o procedimento acima. Diz que árvore é escolhida e que valor dá:

- **(a)** `8-4-2` com `%right '-'`.
- **(b)** `1+2*3` com as declarações por esta ordem: `%nonassoc '<'`, `%left '*'`, `%left '+' '-'` (repara na ordem!).
- **(c)** `1<2+3` e `1<2<3`, com as mesmas declarações da (b). Para a comparação, conta verdadeiro como 1 e falso como 0.

Soluções: Ex. B (a), Ex. B (b), Ex. B (c)

Pratica agora **Calculadora com Flex/Bison ou Alex/Happy (Folha lab. 4, Exercício 1)**

Ficheiros em `Praticas/Semana_3/Parsing/` (`srcC` ou `srcHaskell`, e os testes em `testsC`/`testsHaskell`). Em C precisas do Bison 3 (`brew install bison`, fica em `/opt/homebrew/opt/bison/bin`): o do macOS é o 2.3 e não aceita `%define api.value.type`. O Happy e o Alex instalam-se com `cabal install happy alex`.
Ex. 1(a) Completa o *lexer* e o *parser* para a gramática ambígua $E \to E{+}E \mid E{-}E \mid E{*}E \mid E/E \mid (E) \mid$ `num`, calculando o valor durante a análise (como em "Calcular o resultado"). Resolve as ambiguidades com `%left` (dois níveis de precedência). Usa vírgula flutuante (`float`/`Float`): o `num` tem de aceitar `1.5`. Confirma `1+2*3` $= 7$, `1+2/3` ≈ 1.667, `1-2-3` $= -4$ e `1/2/3` ≈ 0.167, e corre os testes.
Ex. 1(b) Acrescenta $E \to F(E)$ com $F \to$ `sqrt` | `exp` | `log`. Pistas: as palavras `sqrt`/`exp`/`log` são *tokens* novos no *lexer* (a ordem das regras importa? há identificadores aqui?). Em C, `sqrtf`, `expf`, `logf` de `<math.h>` (o `Makefile` já liga com `-lm`). O `input5.txt` testa isto.
Soluções: 1 (a), 1 (b)

Sintaxe concreta e abstrata

A árvore de derivação tem coisas que **não interessam** ao resto do compilador: **parêntesis** de agrupamento, **palavras reservadas** que só delimitam partes (`then`, `else`, `begin`/`end`) e **não-terminais que só servem para desambiguar** (o T e o F das expressões).

Definição **Sintaxe concreta e sintaxe abstrata**

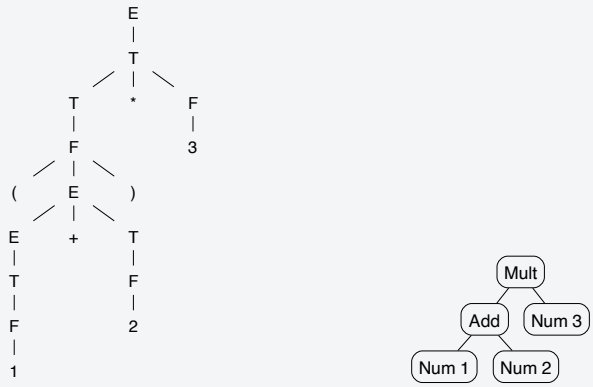
- **Árvore sintática concreta** (de derivação): um nó por cada símbolo de cada produção usada.
- **Árvore sintática abstrata** (AST): só a **estrutura essencial**, com **um nó por operação** (ou construção da linguagem) e os operandos como filhos.

As ações semânticas constroem logo a AST; a árvore concreta nunca chega a existir em memória.

Procedimento para passar da concreta à AST †: (1) apaga os parêntesis e as palavras reservadas que só separam; (2) apaga as cadeias de um filho só ($E \to T \to F$); (3) cada operador ou construção passa a **um nó** com os operandos como filhos, pela mesma ordem.

Exemplo resolvido **(1+2)*3 com a gramática desambiguada**

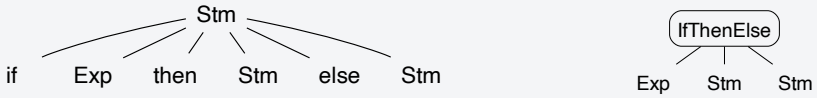
$E \to E + T \mid T$, $T \to T * F \mid F$, $F \to$ `num` $\mid (E)$. Derivação: $E \Rightarrow T \Rightarrow T * F \Rightarrow F * F \Rightarrow (E) * F \Rightarrow \cdots \Rightarrow$ `(1+2)*3`.



Árvore concreta (17 nós) e AST (5 nós). As cadeias $E \to T \to F$ só impõem a precedência e os parêntesis só agrupam. A AST tem um nó por operação (`Mult`, `Add`) e um por constante; o agrupamento `(1+2)` está na **forma** da árvore (o `Add` é filho do `Mult`).

Exemplo resolvido **Comandos: if-then-else**

Stm → `if` *Exp* `then` *Stm* `else` *Stm* | ... Na árvore concreta cada palavra reservada é um nó; na AST `if/then/else` é **um só nó** com três filhos:



`then` e `else` só servem ao analisador para saber onde acaba cada parte; depois, a posição do filho já o diz.

Para que serve: as fases seguintes ficam mais simples e menos ligadas à linguagem-fonte, e o **açúcar sintático** desfaz-se logo no analisador (ex.: $x > y \longrightarrow y < x$ e $x >= y \longrightarrow y <= x$, e o resto do compilador só implementa 2 comparações em vez de 4).

Representar sintaxe abstrata

O compilador tem de **construir** ASTs (nas ações) e **processá-las** (uma função com um caso por construção). O exemplo é a linguagem dos **programas sequenciais** (Folha lab. 3, Exercício 2): instruções `ident = exp`, `ident++` e `stm1; stm2`; expressões com identificadores, números e operadores binários. A regra é a mesma em todas as linguagens: **um tipo por categoria sintática, uma alternativa por produção da AST**.

Linguagens funcionais (Haskell, ML, OCaml, F#)

Cada categoria sintática é um **tipo algébrico**; cada alternativa é um **construtor** com os seus atributos (possivelmente recursivos). Distinguem-se por **encaixe de padrões**.

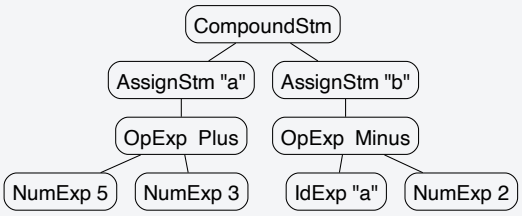
```haskell
data Stm = AssignStm String Exp     -- ident = exp
         | IncrStm String           -- ident++
         | CompoundStm Stm Stm      -- stm1; stm2

data Exp = IdExp String             -- x, y, z, etc.
         | NumExp Int               -- 123, etc.
         | OpExp Exp BinOp Exp      -- e1+e2, e1*e2, ...

data BinOp = Plus | Minus | Times | Div
```

Exemplo resolvido **A AST de a = 5+3; b = a-2**

```haskell
example :: Stm
example =
  CompoundStm
    (AssignStm "a"
       (OpExp (NumExp 5) Plus (NumExp 3)))
    (AssignStm "b"
       (OpExp (IdExp "a") Minus (NumExp 2)))
```



Processa-se com uma função com um caso por construtor, em equações ou com `case`:

```haskell
process :: Stm -> ...
process (IncrStm id)        = ...
process (AssignStm id exp)  = ...
process (CompoundStm s1 s2) = ...
```

```haskell
process stm = case stm of
   IncrStm id        -> ...
   AssignStm id exp  -> ...
   CompoundStm s1 s2 -> ...
```

Linguagem C

Definição **AST em C**

- Uma **`struct`** por categoria sintática, com uma **etiqueta** (*tag*, um `enum` com a alternativa) e uma **`union`** das alternativas; uma alternativa com mais de um atributo é ela própria uma `struct`.
- **Funções construtoras** que reservam memória e preenchem todos os campos.
- **Estilo funcional**: estruturas **imutáveis** e memória **libertada só no fim** (de uma fase ou do programa).
- Processa-se com um **`switch` sobre a *tag***.

```c
struct _stm {
  enum {COMPOUND, ASSIGN, INCR} tag;
  union {
    struct { struct _stm *fst, *snd; } compound;          // COMPOUND
    struct { char *ident; struct _exp *expr; } assign;    // ASSIGN
    char *ident;                                          // INCR
  };
};
typedef struct _stm *Stm;
```

```c
typedef enum {PLUS, MINUS, TIMES, DIV} binop;
struct _exp {
  enum {ID, NUM, OP} tag;
  union {
    int val;                                              // NUM
    char *id;                                             // ID
    struct { binop op; struct _exp *left, *right; } binop;// OP
  };
};
typedef struct _exp *Exp;
```

```c
Exp mk_num(int v) {
  Exp e = (Exp) malloc(sizeof(struct _exp));
  e->tag = NUM;
  e->val = v;
  return e;
}

Exp mk_ident(char *txt) {
  Exp e = (Exp) malloc(sizeof(struct _exp));
  char *str = malloc(strlen(txt)+1);
  strcpy(str, txt);     // "clonar" a cadeia
  e->tag = ID;
  e->id = str;
  return e;
}

Exp mk_op(binop op, Exp e1, Exp e2) {
  Exp e = (Exp) malloc(sizeof(struct _exp));
  e->tag = OP;
  e->binop.op = op;
  e->binop.left = e1;
  e->binop.right = e2;
  return e;
}
```

(E o mesmo para `mk_assign`, `mk_incr`, `mk_compound`.)

Erro típico: guardar o `yytext` sem o copiar †. O Flex reescreve o `yytext` a cada *token*, e o *reduce* pode acontecer **depois** de o Flex já ter lido mais *tokens* (o *look-ahead*, ou toda a expressão em `ID '=' expr`). Nessa altura o ponteiro guardado já aponta para outro texto (do tipo `"x = 10;"`). Por isso copia-se **logo no Flex**: `yylval.str = strdup(yytext);` †.

```
Stm example =
    mk_compound
    (mk_assign("a", mk_op(PLUS, mk_num(5), mk_num(3))),
     mk_assign("b", mk_op(TIMES, mk_ident("a"), mk_num(2))));
```

Nas ações do Bison os valores **não têm todos o mesmo tipo** (`TOK_NUM` é `int`, `exp/term` são `Exp`), por isso usa-se `%union` em vez de `%define api.value.type` †, e no Flex escreve-se no campo certo (`yylval.num = atoi(yytext);`):

```
%union { int num; Exp exp; }
%token <num> TOK_NUM
%type  <exp> exp term
%%
exp : term             { $$ = $1; }
    | exp '+' term     { $$ = mk_op(PLUS, $1, $3); }
    ;
term : TOK_NUM         { $$ = mk_num($1); }
     | '(' exp ')'     { $$ = $2; }
     ;
```

Para processar, um `switch` sobre a *tag*; cada caso só deve usar os campos da sua alternativa, porque o C não o impede:

```
void process(Stm stm) {
  switch(stm->tag) {
  case COMPOUND: /* stm->compound.fst e stm->compound.snd */ break;
  case ASSIGN:   /* stm->assign.ident e stm->assign.expr  */ break;
  case INCR:     /* stm->ident                             */ break;
  }
}
```

O lab de ASTs (`Praticas/Semana_4/Abstract_Syntax_Trees`) usa exatamente este esquema: o `Expr` tem a *tag* `kind` (`E_INTEGER` ou `E_OPERATION`) e a `union attr` (`value` ou `op`, com o `operator` e os ramos `left/right`). O Bison guarda a árvore na global `root` e o `main` chama `eval(root)`, um `switch` recursivo sobre a *tag*. O esqueleto, tal como vem, não liga (`Undefined symbols: _root`): o `parser.bison` declara `extern Expr* root;` mas nenhum ficheiro a define. Basta acrescentar `Expr* root = NULL;` depois do segundo `%%` do `parser.bison` †.

Exemplo resolvido **Lab. ASTs: o esqueleto com 4 + 3 + 2 + 1 (example2.txt)**

Flex: a linha `# A sum of 4 numbers` é comentário (`#.*\n`) e os espaços são ignorados. Tokens: `INT(4) PLUS INT(3) PLUS INT(2) PLUS INT(1)`.

Bison, com `%left PLUS`:

1. shift `INT(4)` e reduz `expr → INT`: `ast_integer(4)` = nó a.
2. shift `PLUS` e `INT(3)` e reduz `expr → INT`: nó b = `ast_integer(3)`.
3. Pilha `expr PLUS expr`, *look-ahead* `PLUS`: conflito *shift/reduce* entre regra e *token* com a mesma precedência, e `%left` manda **reduzir**: `ast_operation(PLUS, a, b)` = nó c ($4 + 3$).
4. shift `PLUS` e `INT(2)` e reduz: nó d = `ast_integer(2)`. *Look-ahead* `PLUS`, reduz outra vez: nó e = `ast_operation(PLUS, c, d)`.
5. shift `PLUS` e `INT(1)` e reduz: nó f = `ast_integer(1)`. *Look-ahead* fim, reduz: nó g = `ast_operation(PLUS, e, f)`.
6. `program → expr`: `root = g`.

Árvore (associativa à esquerda): `((4 + 3) + 2) + 1`.

`eval(g)`: `kind == E_OPERATION`, por isso calcula os dois ramos:

- `eval(e)` é `eval(c) + eval(d)`, com `eval(c)` = `eval(a) + eval(b)` = $4+3 = 7$ e `eval(d)` = 2, portanto `eval(e)` = $7 + 2 = 9$;
- `eval(f)` = 1;
- `operator == PLUS`, portanto o resultado é $9 + 1 = 10$.

Sai `Result = 10`.

Java e outras linguagens

Em **Java** (OO): uma **classe abstrata** por categoria (`abstract class Stm`), uma **subclasse** por alternativa com um construtor que inicializa os atributos (`class AssignStm extends Stm { String id; Exp exp; ... }`), objetos imutáveis; processa-se com uma cadeia de **`instanceof`** + ***downcast*** (ou com o padrão *visitor*). Scala, Kotlin, Swift e Rust têm tipos algébricos e encaixe de padrões. **Para ASTs prefere-se a abordagem funcional**: o compilador avisa se faltar um caso; em C é preciso disciplina com a `union`, e em Java ficam os *casts*.

Pratica agora **Construir e processar ASTs (lab. ASTs e exercícios inventados)**

Lab em `Praticas/Semana_4/Abstract_Syntax_Trees` (C, Flex e Bison 3: `PATH=/opt/homebrew/opt/bison/bin:$PATH make`). Antes de mais, define o `root` em falta (ver acima).

Lab. ASTs, Ex. 2 Acrescenta `-`, `*`, `/` e `%` (*tokens* no Flex, `%left` em dois níveis, regras de `expr`, casos no `eval`). Escreve `printExpr(Expr*)` num `printAbsTree.c` novo e chama-a no `main`. O `example3.txt` tem de dar 999. Depois experimenta `4-1` sem espaços. O erro vem da regra `\-?[0-9]+` (*longest match*, Aula 3).

Lab. ASTs, Ex. 3 Expressões booleanas: um inteiro (0/1) ou `expr <relop> expr`, com `==`, `!=`, `<`, `>`, `<=` e `>=`. Define o tipo `BoolExpr` e os seus construtores, os *tokens* e as regras, e `printBool(BoolExpr*)`. Atenção ao Flex: o `<=` tem de ganhar ao `<`.

Lab. ASTs, Ex. 4 Comandos: atribuição, `if-then`, `if-then-else`, `while`, sequência `c1 ; c2` e blocos `{ }`. Define o tipo `Cmd`, os construtores e as regras (a raiz passa a ser um comando), e `printCmd(Cmd*)`. Pistas: a atribuição precisa de **variáveis** nas expressões, que o enunciado não pede (o `ID` e o `E_VAR` †). Copia o `yytext` no Flex. O `else` pendente e o `;` resolvem-se com precedências, como em "Conflitos, associatividade e precedência".

Ex. C Escreve a AST do programa `x = 1; y = x * (2 + 3); x++`, agrupando as instruções à direita (`x = 1; (y = ...; x++)`):

- **(a)** como valor Haskell, com os tipos `Stm`/`Exp`/`BinOp` acima;
- **(b)** em C, com os construtores `mk_...` (escreve também `mk_assign`, `mk_incr` e `mk_compound`, que os slides não mostram).

Ex. D Escreve um **interpretador** em Haskell para a linguagem dos programas sequenciais: `exec :: Env -> Stm -> Env`, com `type Env = [(String, Int)]` (o valor atual de cada variável). Usa encaixe de padrões, como no `process` acima, e uma função auxiliar `evalExp :: Env -> Exp -> Int`. Testa com o programa do Ex. C: no fim deve dar `x = 2` e `y = 5`.

Soluções: ASTs 2, ASTs 3, ASTs 4, Ex. C (a), Ex. C (b), Ex. D

7 Aula 7
Análise sintática *top-down* (LL)

A análise ***top-down*** constrói a árvore a partir da **raiz** (o símbolo inicial) até às folhas, ao contrário da LR. Escreve-se à mão como **descida recursiva**, ou com uma **tabela** e uma pilha (*parsing* preditivo). Nos dois casos a gramática tem de deixar **escolher a produção só pelo próximo *token***: são as gramáticas **LL(1)**.

Descida recursiva

> **Definição** **Parsing por descida recursiva**
>
> O analisador escreve-se diretamente numa linguagem de programação:
>
> - cada **não-terminal** é uma **função**;
> - cada **produção** é um **caso** dessa função, escolhido pelo **próximo terminal** (o *look-ahead*);
> - dentro de um caso, os símbolos do lado direito tratam-se por ordem: um terminal **consome-se** (tem de ser o próximo, senão é erro), um não-terminal **chama** a sua função.

Procedimento para escrever o analisador:

1. **Cada produção de X começa por um terminal diferente?** Se não, a gramática tem de ser transformada primeiro (secção seguinte).
2. **Uma função `parseX` por não-terminal**, com um caso por produção, escolhido pelo primeiro *token*.
3. **Em cada caso**, percorre o lado direito: terminal → `consume`; não-terminal → chama `parseY`.
4. **Marcador do final:** no fim, verifica que não sobram *tokens*.

Erros típicos: voltar a consumir o primeiro terminal, que o `case` já viu; esquecer o marcador do final (aceitar `print 1=1 end`).

O exemplo dos slides reconhece "programas" como este:

```
begin
  if 1=1 then begin print 0=11 ; print 123=4 end
  else print 11=42
end
```

$$S \to \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{begin}\ S\ L \mid \texttt{print}\ E \qquad L \to \texttt{end} \mid \texttt{;}\ S\ L \qquad E \to \texttt{num} = \texttt{num}$$

Passo 1: as produções de S começam por `if`, `begin`, `print`, e as de L por `end`, `;`: todas diferentes. Pode escrever-se já.

Em Haskell

Cada `parseX :: [Token] -> [Token]` recebe a lista de *tokens* e devolve os que **não consumiu** (ou aborta com `error`). Só **reconhece** a frase, não constrói a árvore. O `deriving (Eq, Show)` † é o que permite o `==` e o `show` do `consume`.

```haskell
data Token = IF | THEN | ELSE | BEGIN | END | PRINT | SEMI | NUM | EQUAL
  deriving (Eq, Show)

consume :: Token -> [Token] -> [Token]       -- consome um token ou aborta
consume tok (first:rest) | tok == first = rest
consume tok _ = error ("expected " ++ show tok)

parseS, parseL, parseE :: [Token] -> [Token]
parseS (first:toks) = case first of
  IF    -> let toks1 = parseE toks           -- S -> if E then S else S
               toks2 = consume THEN toks1
```

```
               toks3 = parseS toks2
               toks4 = consume ELSE toks3
           in parseS toks4
  BEGIN -> let toks1 = parseS toks          -- S -> begin S L
           in parseL toks1
  PRINT -> parseE toks                      -- S -> print E
  _     -> error "syntax error"

parseL (first:toks) = case first of
  END  -> toks                              -- L -> end
  SEMI -> let toks1 = parseS toks           -- L -> ; S L
          in parseL toks1
  _    -> error "syntax error"

parseE toks = let toks1 = consume NUM toks  -- E -> num = num
                  toks2 = consume EQUAL toks1
              in consume NUM toks2

accepted :: [Token] -> Bool                 -- marcador do final
accepted toks = null (parseS toks)
```

A lista que sai de uma função entra na seguinte (`toks1`, `toks2`, …). O padrão `(first:toks)` já tirou o primeiro terminal, por isso não se volta a consumir. Sem o `accepted`, `print 1=1 end` passava (o `parseS` devolve `[END]`). A alternativa é um *token* \$ (fim do ficheiro) e uma produção $S' \to S\,\$$.

Exemplo resolvido **Traçar `accepted` em `begin print 1=1 ; print 2=3 end`**

Tokens: `[BEGIN, PRINT, NUM, EQUAL, NUM, SEMI, PRINT, NUM, EQUAL, NUM, END]`.

1. `parseS` vê `BEGIN`: produção $S \to \texttt{begin}\,S\,L$. Chama `parseS` com o resto, `[PRINT, NUM, EQUAL, NUM, SEMI, PRINT, NUM, EQUAL, NUM, END]`.
2. Esse `parseS` vê `PRINT`: $S \to \texttt{print}\,E$, chama `parseE` com `[NUM, EQUAL, NUM, SEMI, ...]`.
3. `parseE`: `consume NUM` deixa `[EQUAL, NUM, SEMI, ...]`; `consume EQUAL` deixa `[NUM, SEMI, ...]`; `consume NUM` deixa `[SEMI, PRINT, NUM, EQUAL, NUM, END]`. É o `toks1` do passo 1.
4. `parseL [SEMI, PRINT, NUM, EQUAL, NUM, END]` vê `SEMI`: $L \to \texttt{;}\,S\,L$, chama `parseS [PRINT, NUM, EQUAL, NUM, END]`.
5. Vê `PRINT`: `parseE [NUM, EQUAL, NUM, END]` consome os três e devolve `[END]`.
6. `parseL [END]` vê `END`: $L \to \texttt{end}$, devolve `[]`.
7. Todas as chamadas devolvem `[]`, e `accepted` dá `null [] = True`.

A pilha de chamadas (`parseS` → `parseS` → `parseE`, depois `parseL` → `parseS`/`parseL`) segue a árvore sintática da raiz para as folhas.

Em C

As funções **consomem os *tokens* da entrada** (`Token getToken(void)`, o *lexer*) em vez de transformarem listas; o *token* de *look-ahead* fica numa global. Aqui o `case` não consome nada, por isso cada caso começa por `advance()`:

```
Token next;                                 // próximo token (global)
void advance(void) { next = getToken(); }   // avançar um token
void consume(Token t) {                     // consumir um token específico †
  if (next == t) advance(); else error("syntax error");
}

void parse_S(void) {
  switch (next) {
```

```
  case IF:    advance(); parse_E(); consume(THEN); parse_S();
              consume(ELSE); parse_S(); break;
  case BEGIN: advance(); parse_S(); parse_L(); break;
  case PRINT: advance(); parse_E(); break;
  default:    error("syntax error");
  }
}
void parse_E(void) { consume(NUM); consume(EQUAL); consume(NUM); }
void parse_L(void) {
  switch (next) {
  case END:  advance(); break;
  case SEMI: advance(); parse_S(); parse_L(); break;
  default:   error("syntax error");
  }
}
void accepted(void) { parse_S(); consume(EOF); }   // marcador do final
```

Antes da primeira chamada faz-se um `advance()` † para carregar o primeiro *token* em `next`.

Pratica agora **Descida recursiva (exercícios inventados)**

Ex. E Com o analisador Haskell acima:

- **(a)** traça `accepted` em `if 1=1 then print 2=3 else begin print 4=4 end`, mostrando a lista que cada chamada recebe e devolve;
- **(b)** diz onde e porquê falha `begin print 1=1 print 2=2 end`;
- **(c)** diz o que dá `parseS` e `accepted` em `print 1=1 end`.

Ex. F Completa a versão em C num programa que corre: um `getToken` que lê de um *array* de *tokens* terminado em `EOF`, o `error` e o `main`. Testa com `begin print 1=1 ; print 2=3 end` e com `print 1=1 end`. Pista: no `enum`, `EOF` já é uma macro do `stdio.h`; dá-lhe outro nome.

Soluções: Ex. E (a), Ex. E (b), Ex. E (c), Ex. F

Gramáticas LL(1) e *parsing* preditivo

A gramática das expressões

$$E \to E + T \mid E - T \mid T \qquad T \to T * F \mid T/F \mid F \qquad F \to \texttt{num} \mid (E)$$

falha o passo 1 da descida recursiva de duas maneiras:

- as três produções de E (e as de T) começam todas por `num` ou `(`: o próximo símbolo não diz qual usar;
- a **recursão à esquerda** entra em ciclo: `parseE` chamaria logo `parseE`, sem consumir nada.

Transformar a gramática

Definição **Eliminar recursão à esquerda**

$A \to A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$ (nenhum β_i começa por A) passa a

$$A \to \beta_1 A' \mid \cdots \mid \beta_n A' \qquad A' \to \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon$$

(regra geral †). A linguagem é a mesma: primeiro um β, depois zero ou mais α. A recursão passa a ser **à direita**, e A' decide pelo próximo símbolo: com o início de um α continua, senão usa ε.

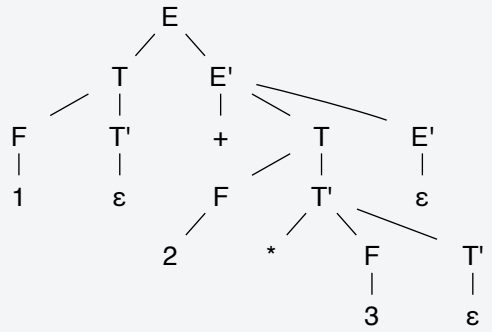
Procedimento: (1) separa as produções de A em **recursivas** ($A \to A\alpha$: fica o α) e **não recursivas** ($A \to \beta$); (2) cada β passa a $\beta A'$; (3) cada α passa a $\alpha A'$; (4) junta $A' \to \varepsilon$.

Erro típico: esquecer $A' \to \varepsilon$ (a repetição nunca pára) ou pôr o A' à esquerda ($A' \to A'\alpha$, que volta a ser recursão à esquerda).

Exemplo resolvido

$E \to E + T \mid E - T \mid T$: recursivas com $\alpha = +T$ e $\alpha = -T$; não recursiva com $\beta = T$. Dá $E \to T\,E'$ e $E' \to +\,T\,E' \mid -\,T\,E' \mid \varepsilon$. O mesmo para T:

$$E \to T\,E' \qquad T \to F\,T' \qquad F \to \texttt{num}$$
$$E' \to +\,T\,E' \mid -\,T\,E' \mid \varepsilon \qquad T' \to *\,F\,T' \mid /\,F\,T' \mid \varepsilon \qquad F \to (E)$$



Árvore de `1 + 2 * 3` na gramática transformada. A precedência mantém-se (o $*$ fica mais fundo, dentro do T do 2), mas a árvore deixa de mostrar a associatividade à esquerda: em `1 - 2 - 3` os E' encadeiam-se para a direita. Ao construir a AST, passa-se o valor já acumulado à esquerda para dentro de E', para recuperar $(1-2)-3$ (†).

Fatorização à esquerda †. Duas produções com o mesmo prefixo, $A \to \alpha\beta_1 \mid \alpha\beta_2$, também impedem a escolha. Adia-se a decisão para depois do prefixo: $A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$.

A classe LL(1)

Definição **Gramática LL(1)**

LL(1) = *Left-to-right parse, Leftmost derivation, 1-symbol look-ahead*: as gramáticas que se analisam por descida recursiva (ou com a tabela abaixo). Com NULLABLE, FIRST e FOLLOW (Aula 5), com o próximo terminal x, usa-se $X \to \gamma$ se:

- $x \in \text{FIRST}(\gamma)$; ou
- γ é anulável e $x \in \text{FOLLOW}(X)$ (o X desaparece e o x é o que vem a seguir).

É LL(1) se, para cada X, **nenhum terminal serve duas produções**.

Uma gramática com recursão à esquerda **nunca** é LL(1) †: FIRST($A\alpha$) contém FIRST(β), e com esse terminal servem as duas produções. A das expressões original não é (FIRST($E+T$) = FIRST(T) = {num, (}); a transformada é (Ex. H).

Definição **Tabela de parsing preditivo (LL(1))**

Linhas = não-terminais; colunas = terminais (com o \$). Cada produção $X \to \gamma$ vai para:

- a linha X, nas colunas $t \in \text{FIRST}(\gamma)$;
- se NULLABLE(γ), também nas colunas $t \in \text{FOLLOW}(X)$.

A gramática é **LL(1) se e só se cada entrada tiver no máximo uma produção**. Uma entrada vazia é erro sintático.

Procedimento para decidir se é LL(1) (e construir a tabela):

1. **Há recursão à esquerda?** Então não é; elimina-a.
2. **Há produções do mesmo X com o mesmo prefixo?** Fatoriza.
3. **Calcula NULLABLE, FIRST e FOLLOW** (com $S' \to S\,\$$).

4. **Para cada produção**, as colunas do FIRST do lado direito, e as do FOLLOW do lado esquerdo se o lado direito for anulável.
5. **Alguma entrada com duas produções?** Não é LL(1).

Erro típico: pôr uma produção $X \to \varepsilon$ em nenhuma coluna (o FIRST de ε é vazio): ela vai para as colunas de **FOLLOW(X)**.

Exemplo resolvido **NULLABLE, FIRST e FOLLOW de S para AB (exercício dos slides)**

$S' \to S\,\$$, $S \to AB$, $A \to aAb \mid \varepsilon$, $B \to bB \mid \varepsilon$ (cada iteração usa a coluna anterior). Passos 1 e 2: não há recursão à esquerda nem prefixos comuns.
NULLABLE: $N(S) = N(A) \wedge N(B)$; $N(A) = (N(a) \wedge N(A) \wedge N(b)) \vee N(\varepsilon) =$ True; $N(B) = (N(b) \wedge N(B)) \vee$ True = True; $N(S') = N(S) \wedge N(\$) =$ False. Iteração 1: A e B True, S ainda False (A, B eram False na 0). Iteração 2: $S =$ True $\wedge$ True = True. Iteração 3: igual. **NULLABLE(S) = NULLABLE(A) = NULLABLE(B) = True.**
FIRST: FIRST(S) = FIRST(A) $\cup$ FIRST(B) (A anulável); FIRST(A) = $\{a\} \cup \varnothing$; FIRST(B) = $\{b\} \cup \varnothing$. Iteração 1: FIRST(A) = $\{a\}$, FIRST(B) = $\{b\}$; iteração 2: FIRST(S) = $\{a, b\}$; iteração 3: igual. **FIRST(S) = $\{a, b\}$, FIRST(A) = $\{a\}$, FIRST(B) = $\{b\}$.**
FOLLOW, ocorrência a ocorrência:

Produção	Ocorrência	Contribuição
$S' \to S\,\$$	S, $\beta = \$$	FOLLOW(S) $\supseteq$ FIRST(\$) = {\$}
$S \to AB$	A, $\beta = B$	FOLLOW(A) $\supseteq$ FIRST(B) = $\{b\}$; B anulável: FOLLOW(A) $\supseteq$ FOLLOW(S)
$S \to AB$	B, $\beta = \varepsilon$	FOLLOW(B) $\supseteq$ FOLLOW(S)
$A \to aAb$	A, $\beta = b$	FOLLOW(A) $\supseteq$ $\{b\}$
$B \to bB$	B, $\beta = \varepsilon$	FOLLOW(B) $\supseteq$ FOLLOW(B) (nada)

	0	1	2	3
FOLLOW(S)	$\varnothing$	{\$}	{\$}	{\$}
FOLLOW(A)	$\varnothing$	$\{b\}$	$\{b, \$\}$	$\{b, \$\}$
FOLLOW(B)	$\varnothing$	$\varnothing$	{\$}	{\$}

Na 1, FOLLOW(A) e FOLLOW(B) recebem o FOLLOW(S) da coluna 0 ($\varnothing$); só na 2 recebem o \$.

Exemplo resolvido **Tabela de S para AB**

Passo 4, produção a produção:

- $S' \to S\,\$$: S anulável, FIRST$(S\,\$) = \{a, b\} \cup \{\$\}$: colunas a, b, \$.
- $S \to AB$: FIRST$(AB) = \{a\} \cup \{b\}$ (A anulável), e AB é anulável, por isso também FOLLOW$(S) = \{\$\}$: colunas a, b, \$.
- $A \to aAb$: coluna a. $A \to \varepsilon$: FIRST vazio, anulável: colunas de FOLLOW$(A) = \{b, \$\}$.
- $B \to bB$: coluna b. $B \to \varepsilon$: FOLLOW$(B) = \{\$\}$.

	a	b	\$
S'	$S' \to S\,\$$	$S' \to S\,\$$	$S' \to S\,\$$
S	$S \to AB$	$S \to AB$	$S \to AB$
A	$A \to aAb$	$A \to \varepsilon$	$A \to \varepsilon$
B		$B \to bB$	$B \to \varepsilon$

Passo 5: no máximo uma produção por entrada: **a gramática é LL(1)**.

Parsing preditivo com pilha explícita

A descida recursiva usa a pilha de chamadas da linguagem. Com a tabela faz-se o mesmo **sem recursão**, com uma pilha dos símbolos que **ainda faltam reconhecer**:

```
stack := empty; push(S', stack);
while (stack not empty) do
  if top(stack) is a terminal then          /* consumir input */
    consume(top(stack)); pop(stack);
  else if table[top(stack), next] is empty then
    report_error();
  else                                       /* usar uma produção */
    symbols := right_hand_side(table[top(stack), next]);
    pop(stack);
    pushList(symbols, stack);                /* 1.º símbolo fica no topo */
```

Procedimento (cada passo): **o topo é um terminal?** Tem de ser igual ao próximo da entrada: consome-se e tira-se. **É um não-terminal X?** Lê (linha X, coluna do próximo); tira X e põe o lado direito, com o **primeiro símbolo no topo**. As produções usadas, por ordem, formam a **derivação mais à esquerda** da entrada (o segundo L de LL, †).

Erro típico: empilhar o lado direito ao contrário (com o último símbolo no topo).

Exemplo resolvido **Parsing de aabbb com a tabela de S para AB**

Pilha com o topo à esquerda.

Pilha	Entrada	Ação
S'	`aabbb$`	$S' \to S\,\$$
$S\,\$$	`aabbb$`	$S \to AB$
$A\,B\,\$$	`aabbb$`	$A \to aAb$ (linha A, coluna a)
$a\,A\,b\,B\,\$$	`aabbb$`	consumir a
$A\,b\,B\,\$$	`abbb$`	$A \to aAb$
$a\,A\,b\,b\,B\,\$$	`abbb$`	consumir a
$A\,b\,b\,B\,\$$	`bbb$`	$A \to \varepsilon$ (coluna b)
$b\,b\,B\,\$$	`bbb$`	consumir b
$b\,B\,\$$	`bb$`	consumir b
$B\,\$$	`b$`	$B \to bB$
$b\,B\,\$$	`b$`	consumir b
$B\,\$$	`$`	$B \to \varepsilon$ (coluna \$)
\$	`$`	consumir \$
ε	ε	aceitação

Pratica agora **Gramáticas LL(1) (exercício dos slides e inventados)**

Ex. G (Exercício dos slides) Implementa em Haskell o analisador por descida recursiva da gramática das expressões **transformada** (uma função por não-terminal, `accepted` no fim). Testa com `1+2*3`, `1*(2-3)/4`, `1+)` e `1+`. Pista: em E' e T', o caso "outro símbolo" é a produção ε (devolve a lista intacta).

Ex. H Para a gramática transformada, com $S' \to E\,\$$:

- **(a)** escreve as equações e calcula NULLABLE, FIRST e FOLLOW pelo método iterativo;
- **(b)** constrói a tabela de *parsing* preditivo e diz se é LL(1).

Ex. I Com a tabela do Ex. H, faz o *parsing* de `num + num * num` com a pilha explícita.

Ex. J $S \to aSb \mid ab$:

- **(a)** mostra que não é LL(1);
- **(b)** fatoriza-a à esquerda, mostra que o resultado é LL(1) e faz o *parsing* de `aabb` com a tabela.

Soluções: Ex. G, Ex. H (a), Ex. H (b), Ex. I, Ex. J (a), Ex. J (b)